BookForMe: An Agentic AI-Powered Centralized Booking Platform

Kaavish Report

presented to the academic faculty

by

Muhammad Taqi	mt08073
Muhammad Ahmad Humayun Hanif	mh08072
Jazib Waqas	jw08048
Taha Hunaid Ali	ta08451

In partial fulfillment of the requirements for

Bachelor of Science

Computer Science

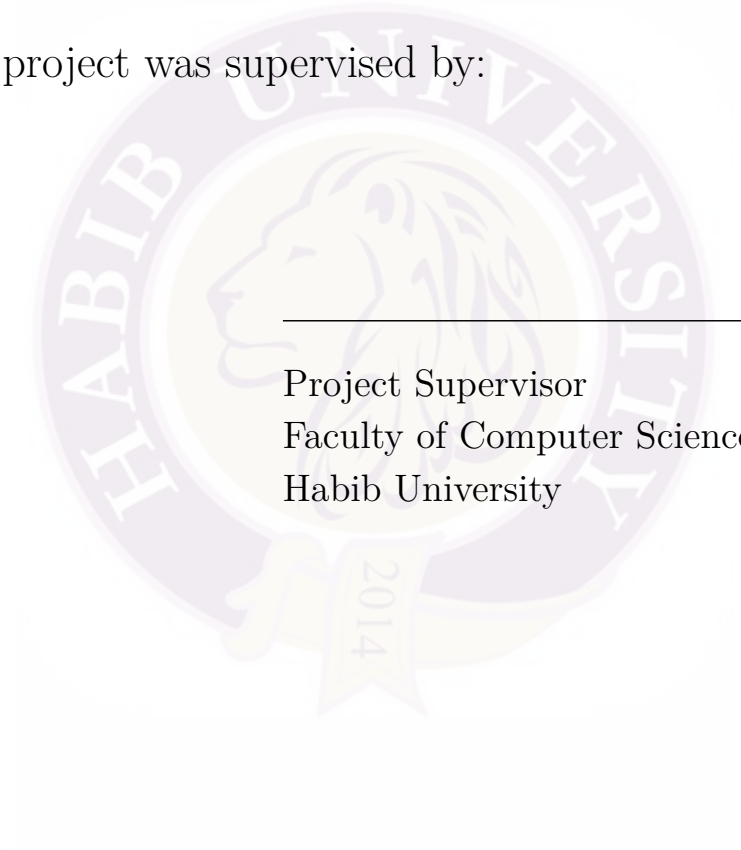
Dhanani School of Science and Engineering

Habib University

Spring 2026

BookForMe: An Agentic AI-Powered Centralized Booking Platform

This Kaavish project was supervised by:



Project Supervisor
Faculty of Computer Science
Habib University

Approved by the Faculty of Computer Science on _____

Acknowledgements

We want to thank the CS faculty at Habib University for their guidance throughout the Kaavish programme, particularly during milestone reviews where their feedback sharpened our system design and evaluation methodology.

We are grateful to the futsal and padel court operators in Karachi who generously shared their WhatsApp booking conversations and allowed us to observe real scheduling workflows. Without their cooperation, our bilingual dataset would not exist.

Finally, we acknowledge the open-source communities behind HuggingFace Transformers, LangGraph, FastAPI, and React Native, whose work made this project technically feasible within one academic year.

Abstract

Booking recreational and informal services in Pakistan—futsal courts, salons, gaming zones, and beach houses—remains fragmented and inefficient. Customers must individually contact vendors over WhatsApp, wait for manual availability checks, and confirm reservations through unstructured conversation. This process is slow, prone to double-bookings, and poorly suited to users who communicate in Roman Urdu.

BookForMe is a centralized, agentic AI booking platform that addresses these problems. It provides a React Native mobile application for browsing and booking services, and an AI Receptionist that autonomously manages booking requests on the vendor's WhatsApp channel. Core contributions are: (1) a custom bilingual dataset of 657+ annotated utterances in English and Roman Urdu constructed through real vendor conversations and LLM-driven augmentation; (2) a comparative evaluation of six transformer architectures for intent classification, with DistilBERT achieving the best performance of 90.5% accuracy and 91.7% macro-F1; (3) a LangGraph-based cyclic state machine implementing the full booking pipeline with Optimistic Concurrency Control (OCC) in Firestore and OCR-based payment verification; and (4) a social layer with Elo-based matchmaking, community forums, and leaderboards.

A user survey ($n > 50$) found that 72% of respondents rely on WhatsApp or phone calls for bookings and 81% reported frustration with multi-vendor coordination, validating the social relevance of this work. The platform is live at https://jhat-to9p.onrender.com and open-sourced at https://github.com/JazibWaqas/JHAT.

Keywords: Agentic AI, Task-Oriented Dialogue, Bilingual NLU, Roman Urdu, LangGraph, Optimistic Concurrency Control, WhatsApp Business API, DistilBERT.

Contents

List of Figures

List of Tables

1. Introduction

1.1 Problem Statement

Booking recreational and informal services in Pakistan is a deeply fragmented and inefficient process. A customer who wants to reserve a futsal court for Friday evening must: (1) find the vendor's WhatsApp number, (2) send a message asking for availability, (3) wait for a manual reply—sometimes hours later, (4) negotiate a time and price, (5) transfer an advance payment via JazzCash or bank transfer, and (6) send a payment screenshot as confirmation. If the chosen slot was taken by another customer during the wait, the entire cycle restarts with a different vendor.

This friction is not limited to sports courts. Salons, gaming zones, beach houses, and most informal service businesses in Karachi operate identically. No industry-wide booking infrastructure serves this segment. Existing digital solutions are narrow in scope—sports-only apps or salon-specific platforms—and do not reflect the cross-category, bilingual booking behaviour of real users.

A user survey conducted by the team in Spring 2025 ($n > 50$) quantified this friction directly [3]:

- **72%** of respondents rely on WhatsApp or phone calls to make bookings.
- **81%** reported frustration with contacting multiple vendors to find an available slot.
- **68%** said they would switch to an app that could handle the conversation and booking on their behalf.

Research by MoldStud [18] corroborates that automated booking systems "increase efficiency, reduce human error, and improve customer satisfaction." Kumar et al. [13] further show that structured digital workflows reduce errors and missed reservations compared to manual methods. Chatterjee et al. [4] demonstrate through a Turf Booking System study that real-time slot availability and automated conflict resolution improve both user experience and vendor management—but their solution remains domain-specific and does not extend to informal multi-category vendors or support autonomous agent-driven negotiation.

At the vendor side, the problems mirror those of users. Vendors manage incoming WhatsApp requests across multiple simultaneous conversations, maintain schedules in personal notebooks or Google Sheets, and face double-bookings whenever they are offline or slow to reply. Small vendors lack the resources to build dedicated booking systems, causing them to miss revenue and to offer a poor customer experience.

The core computer science problem is therefore: *how can an AI agent understand and autonomously execute bilingual natural-language booking requests in real time, while guaranteeing consistency in a distributed, multi-channel environment?* This decomposes into four sub-problems:

1. **Bilingual NLU:** Parse booking requests in both English and Roman Urdu, handling code-switching ("Mujhe Friday ko 7 baje ke baad paddle court chahiye under 6000 rupees, preferably in defence") and severe orthographic variation (*waqt*, *vakt*, *wkht*, and *tym* are all romanisations of the Urdu word for "time").

2. **Dialogue Management:** Maintain conversation state across asynchronous WhatsApp sessions, handle partial information, manage clarification loops, and route user intent to the correct backend action.

3. **Agentic Execution:** Autonomously check availability, hold slots atomically, verify payment proofs via OCR, and notify vendors—without hallucinating actions or bypassing business rules.

4. **Concurrency Safety:** Prevent race conditions in a distributed NoSQL environment where multiple simultaneous requests may attempt to book the same time slot.

1.2 Proposed Solution

BookForMe is a centralized, agentic AI-powered booking platform that addresses all four sub-problems above. It provides two complementary interaction modes:

1. **React Native Mobile App (Rich Client):** A full-featured mobile application that allows customers to browse vendors by category and location, view real-time availability calendars, select and confirm slots, upload payment proof, and access a social layer (matchmaking, forums, leaderboard).

2. **WhatsApp AI Receptionist (Conversational Client):** An autonomous agent that operates on the vendor's existing WhatsApp Business number. Customers message the vendor in plain English or Roman Urdu; the agent understands their request, checks availability from the Firestore database, holds the slot via Optimistic Concurrency Control, requests a payment screenshot, verifies the amount via OCR, and notifies the vendor for final approval—all without requiring the vendor to participate in the conversation.

Both clients share a single backend and database (the "Single Source of Truth"), meaning that a slot booked via WhatsApp is instantly reflected in the mobile app and on the vendor dashboard. Figure 1.1 gives a high-level overview of this architecture.

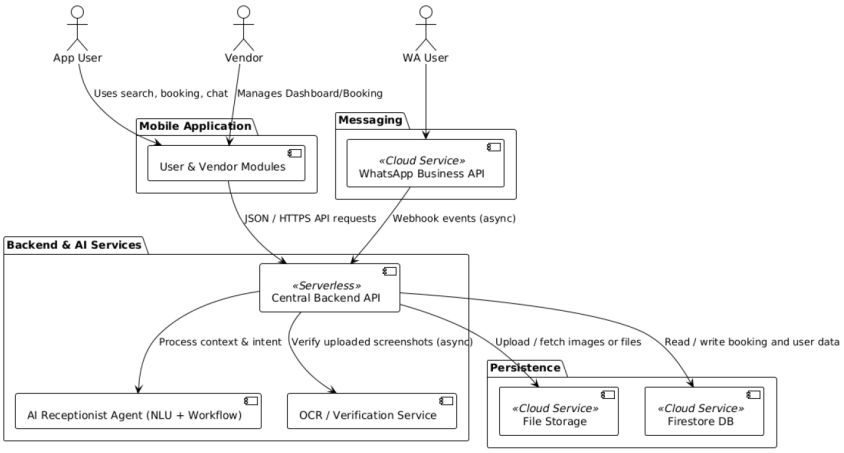


Figure 1.1: High-level overview of the BookForMe platform.

What differentiates BookForMe from existing booking tools is the integration of three capabilities within one platform:

- bilingual NLU tuned to the local mix of English and Roman Urdu,
- a fully autonomous agentic workflow over WhatsApp with transactional safety,
- a social layer for player community and matchmaking across sports, salons, gaming zones, and other informal services.

1.3 Intended User

BookForMe serves three distinct user groups:

App Users (Customers) Residents of Karachi who want to book informal services—predominantly sports-active young adults aged 18–35. They interact with the platform primarily through the React Native mobile app, using the AI chatbot for natural-language search or manual browsing with filters for price, location, and amenities. Social features (player matching, leaderboards, forums) are especially relevant to this group.

WhatsApp Users (Casual Customers) Users who prefer not to install a separate app. They interact exclusively through the vendor's existing WhatsApp Business number. The AI Receptionist handles their entire booking journey—from availability check to payment confirmation—without requiring them to change their communication habits.

Vendors (Small Business Owners) Owners of futsal courts, padel courts, salons, gaming zones, and similar informal services in Karachi. They interact with BookForMe through the Vendor Dashboard, which provides a consolidated booking calendar (aggregating requests from the app and WhatsApp), analytics, pending-booking approvals, and an interface to link their WhatsApp Business account. Crucially, vendors need no technical expertise—onboarding requires only connecting their WhatsApp number.

1.4 Project Gantt Chart and Deliverables

Table 1.1 shows the monthly project timeline across both Kaavish I (CS 491) and Kaavish II (CS 492) semesters.

Kaavish I Deliverables:

- Project proposal (approved)
- SRS v1 and v2
- Software Design Specification (SDS)
- Literature review
- Preliminary bilingual dataset
- Low-fidelity wireframes

Kaavish II Deliverables:

- Functional AI Receptionist (WhatsApp agent, live demo)

Table 1.1: BookForMe project timeline and deliverables.

Month	Tasks	Deliverables
Sep–Oct 2025 (Kaavish I)	Literature review; user survey; initial SRS; system architecture design; team role assignment; GitHub repository setup.	SRS v1; survey results; architecture diagram; project proposal.
November 2025	SRS v2 and SDS; initial bilingual dataset construction; low-fidelity UI wireframes; Firebase project and Firestore schema setup.	SRS v2; SDS; preliminary dataset (200 utterances); wireframes.
December 2025	Baseline NLU models (intent classification); Firestore CRUD APIs; React Native app skeleton (login, home, vendor listing).	Baseline NLU model; defined DB schema; minimal working booking system.
January 2026 (Kaavish II)	Fine-tune NLU pipeline; LangGraph agent implementation; WhatsApp webhook integration; OCC slot locking; OCR payment verification.	Functional AI Receptionist; WhatsApp-to-Firestore data flow; enhanced user dashboard.
February 2026	Vendor dashboard (calendar, analytics, booking approval); social features (matchmaking, forum, leaderboard, chat); model retraining and error analysis.	Full vendor dashboard; active social features; optimised bilingual NLU model.
March 2026	End-to-end integration; unit and integration testing; performance benchmarking; UX testing with real users; mid-evaluation presentation.	Fully integrated platform; test reports; benchmark metrics; mid-eval presentation.
April 2026	Pilot testing with selected Karachi vendors; production deployment on Firebase and Render; final documentation and report.	Production-ready deployment; final report; presentation deck; submission package.

- Full-stack React Native application (user app + vendor dashboard)
- Fine-tuned DistilBERT NLU model (90.5% accuracy)
- OCR payment verification module
- Social features (matchmaking, forum, leaderboard)
- End-to-end test suite and benchmark results
- Production deployment (https://jhat-to9p.onrender.com)
- This final report

1.5 Key Challenges

Roman Urdu Orthographic Variation Roman Urdu lacks a standardised orthography. A single word like *waqt* (time) may appear as *vakt*, *wkht*, *waqat*, or *tym*, depending on the writer. Standard subword tokenisers (BPE, WordPiece) frequently fail to map these variants to a shared representation. We address this through targeted data augmentation and the use of multilingual models that are exposed to diverse romanisation patterns.

Low-Resource Bilingual Dataset No publicly available labelled dataset exists for English/Roman Urdu booking conversations. We constructed one from scratch through real vendor conversations, manual annotation, and LLM-driven synthetic generation. Starting with a small seed set introduces noise and class imbalance, requiring focal loss training and augmentation strategies.

Asynchronous Conversation State WhatsApp conversations are inherently asynchronous: users may reply after hours, change their mind mid-booking, or send the wrong payment screenshot. Unlike synchronous chatbot frameworks, our agent must persist full conversation state in Firestore and resume gracefully. LangGraph's cyclic state graph design addresses this, but requires careful state serialisation.

Concurrency and Double-Booking Multiple users may simultaneously request the same time slot. In a distributed NoSQL database like Firestore, naïve writes allow race conditions. We implement Optimistic Concurrency Control (OCC) using Firestore atomic transactions with a `hold_expires_at` timestamp, but tuning the hold duration (too short causes false failures; too long ties up inventory) required careful calibration.

LLM Latency Calls to the Gemini API for complex slot extraction add 3–10 seconds of latency per message. The total WhatsApp response time budget (NFR-PERF-03) is 60 seconds. We minimise latency by using the fine-tuned DistilBERT model for fast intent classification (47 ms on CPU) and calling the LLM only for slot extraction and edge cases, caching frequent availability queries.

Prompt Injection Security Malicious users may attempt to trick the WhatsApp agent into bypassing payment requirements through carefully crafted messages. The Neuro-Symbolic architecture—where the LLM classifies intent but all database mutations are executed by deterministic code nodes—prevents such attacks, as the LLM cannot directly issue database commands.

2. Literature Review

This chapter presents the current state of the art across the four research areas that underpin BookForMe: (1) agentic AI systems built on large language models; (2) task-oriented dialogue and state tracking; (3) bilingual and code-switched NLU for South Asian languages; and (4) distributed booking systems with concurrency guarantees. It also establishes the novelty of our work by identifying gaps that existing research does not address and that directly motivate our design choices.

2.1 From Chatbots to Autonomous Agents

2.1.1 Defining the Agentic Architecture

The term "agent" has been used broadly in software engineering and robotics, but its application to large language models (LLMs) represents a specific architectural shift. Wang et al. [29] propose a unified framework in which an LLM-based agent comprises four modules: *Profiling* (role and persona definition), *Memory* (short-term context and long-term persistent state), *Planning* (decomposing goals into subgoals and choosing actions), and *Action* (calling external tools and APIs to effect changes in the environment). In this framework the LLM functions as a "Brain" that orchestrates a Perception $\rightarrow$ Reasoning $\rightarrow$ Action cycle, fundamentally distinguishing it from a passive token predictor.

Xi et al. [32] further distinguish agents from traditional conversational systems by their capacity to *initiate and manage workflows* rather than merely respond to prompts. A conventional chatbot can answer "Is the 5 PM slot available?"; our

agent must answer "Book the 5 PM slot, lock it for ten minutes, verify the advance payment, and notify the vendor." The latter requires stateful read–write operations across multiple external systems—precisely the capability gap this work fills.

2.1.2 Cognitive Architectures: ReAct, PRACT, and Reflexion

Yao et al. [33] introduced the **ReAct** (Reasoning + Acting) paradigm, which interleaves chain-of-thought reasoning with external actions, enabling explicit planning traces such as: *Thought*: User requests 5 PM padel. → *Action*: `get_availability("padel","2026-03-15","17:00")`. → *Observation*: Slot available. → *Action*: `create_booking(user, slot)`.

Liu et al. [16] extend this with the **PRACT** agent, adding a self-reflection step *before* executing irreversible actions. Before calling `create_booking()`, the agent verifies that the action is consistent with the user's latest stated intent—a critical safeguard where a confirmed booking constitutes a financial commitment.

Shinn et al. [27] demonstrate in **Reflexion** that verbal reinforcement and persistent state across iterations significantly improve agent reliability on multi-step tasks. The agent receives textual feedback from the environment (e.g., "payment amount incorrect") and updates its policy accordingly without gradient updates. This pattern directly informs our OCR rejection loop, where the agent re-prompts the user upon detecting an incorrect payment amount.

2.1.3 Orchestration Frameworks: LangChain and LangGraph

LangChain [17] provides a modular abstraction layer that decouples application logic from specific LLM providers, enabling composition of prompt templates, vector stores, and output parsers into unified pipelines. However, standard LangChain implementations rely on Directed Acyclic Graphs (DAGs), which cannot represent the iterative, self-correcting loops required for asynchronous WhatsApp conversations where users may reply late, change their mind, or submit incorrect payment proofs.

LangGraph [14] addresses this by replacing DAGs with *Cyclic State Graphs*: the agent's behaviour is modelled as a state machine where nodes represent actions and edges encode conditional logic. The framework provides loop control, state persistence across invocations, and fault recovery— all essential for BookForMe's multi-turn WhatsApp booking conversations. Shang et al. [25] confirm in AgentSquare that modular architectures separating Planning, Reasoning, and Memory are substantially more robust for complex tasks than monolithic prompt designs.

2.1.4 Multi-Agent Decomposition

Händler [9] proposes a multi-dimensional taxonomy of autonomous LLM-powered multi-agent architectures, showing that complex workflows benefit from assigning distinct roles to specialised sub-agents (intent parsing, verification, database updates, external communications) coordinated by a central orchestrator. Wu et al. [30] in AutoGen further argue that effective problem-solving requires cyclic loops—the ability to retry failed actions, request missing information, and self-correct—rather than purely linear pipelines. BookForMe adopts this multi-node, cyclic design in its LangGraph implementation.

2.2 Task-Oriented Dialogue and State Tracking

Task-Oriented Dialogue (TOD) systems have evolved from modular pipelines (NLU → Dialogue State Tracking → Policy → NLG) to end-to-end architectures. **SimpleTOD** [11] treats the entire dialogue as unified causal language modelling, jointly predicting belief states, system actions, and responses in a single autoregressive model. While effective on standard benchmarks (MultiWOZ), it requires large annotated corpora—a significant barrier for our domain where no pre-built dataset exists.

Shin et al. [26] address this data scarcity problem with **DS2** (Dialogue Summaries as Dialogue States), converting conversation history into templated natural-language summaries that map directly to structured slot representations. This summarisation-

based approach is particularly useful for short, noisy WhatsApp messages where maintaining a full belief-state vector is impractical.

2.3 Bilingual and Code-Switched NLU

2.3.1 Roman Urdu Challenges

Processing Roman Urdu introduces challenges largely absent from standard multilingual NLP. Bhat et al. [2] classify Roman Urdu as exhibiting severe *orthographic variation*: a single word can be rendered in multiple phonetically plausible romanisations by different writers (e.g., *waqt*, *vakt*, *wkht*, *waqat*, and *tym* all mean "time"). Standard subword tokenisers (BPE, WordPiece) trained on formal corpora frequently fail to map these variants to a shared semantic representation. Integrating a normalisation layer before intent classification is therefore necessary.

2.3.2 Cross-Lingual Transfer and Adapters

Yu et al. [34] address cross-lingual dialogue state tracking via contrastive learning, aligning slot embeddings across languages so that semantically equivalent slots in different languages map to nearby vector representations. Wu et al. [31] show that parameter-efficient adapter methods can transfer pretrained models to code-switched contexts without full retraining, significantly reducing data and compute requirements.

The Qwen2.5 technical report [22] demonstrates that the 7B variant achieves 79.3 on the MMLU-Multi-Understanding benchmark, the highest among models of comparable size, validating its use as a multilingual backbone.

2.3.3 Multilingual Pretrained Models

Devlin et al. [7] introduced **BERT**, establishing bidirectional masked language modelling as the dominant pretraining paradigm. The multilingual variant (mBERT) was pretrained on 104 languages, including Urdu, providing limited but useful cross-lingual transfer. Conneau et al. [6] demonstrate that **XLM-RoBERTa**, trained on

a substantially larger multilingual corpus with improved data filtering, outperforms mBERT on cross-lingual classification. Sanh et al. [23] show that **DistilBERT**, a distilled 66M-parameter version of BERT, retains 97% of BERT's performance while being 40% smaller and 60% faster—a highly attractive tradeoff for latency-sensitive production deployment.

2.3.4 Data Augmentation for Low-Resource Settings

Given the absence of pre-built bilingual booking datasets, data augmentation is essential. Hou et al. [12] show that back-translation and contextual augmentation improve slot-filling F1 by 6–17% and intent accuracy by up to 12% in booking domains with fewer than 500 annotated utterances. An et al. [1] demonstrate that controllable T5-based generation yields 8–18% absolute gains in intent classification under 10-shot conditions for restaurant and hotel reservation tasks. He et al. [10] in **SynthDST** show that fully synthetic LLM-generated dialogues can surpass real-data baselines, achieving 4–12% higher Joint Goal Accuracy on MultiWOZ when human annotations are extremely limited. BookForMe leverages LLM-driven paraphrasing and synthetic generation following these findings.

2.4 Retrieval, Planning, and Tool Use

Schick et al. [24] demonstrated in **Toolformer** that LLMs can learn to invoke external APIs through self-supervised training, deciding autonomously when and how to call tools—a key capability for our agent's database interactions. Patil et al. [20] (**Gorilla**) and Qin et al. [21] (**ToolLLM**) extend this to controlled invocation of structured APIs, providing the theoretical basis for our `get_availability()`, `create_booking()`, and `get_services()` tool set.

Lewis et al. [15] introduced **Retrieval-Augmented Generation (RAG)**, conditioning generation on dynamically retrieved documents. In BookForMe, RAG enables the agent to access up-to-date vendor availability and pricing from Firestore rather than relying on memorised parametric knowledge—critical for a real-time booking

system. Chen et al. [5] present adaptive retrieval strategies where the agent decides when retrieval is necessary, minimising unnecessary database queries and reducing latency.

2.5 Distributed Booking and Concurrency Safety

Mong'are [19] provides a thorough analysis of idempotency in APIs and distributed systems, specifically addressing the double-booking problem that arises when concurrent requests modify shared mutable state. He recommends idempotent APIs combined with Optimistic Concurrency Control (OCC) or distributed locking as the preferred approach for booking systems. Our implementation directly follows this recommendation, using Firestore atomic transactions with a `hold_expires_at` timestamp to implement OCC.

Singh et al. [28] demonstrate the effectiveness of reinforcement learning for dialogue policy optimisation in the NJFun system, working with compressed 62-state state spaces to "greatly reduce" training data requirements—an approach that validates BookForMe's data-efficient learning strategy for asynchronous multi-party interactions.

2.6 Novelty and Gap Analysis

Table 2.1 summarises the specific gaps that BookForMe addresses relative to the existing literature and systems.

Table 2.1: Gap analysis: BookForMe versus existing dialogue and booking paradigms.

System	Focus	Tool Use	Roman Urdu	Async Mem.	Txn. Safety
GPT / General LLM	General chat	✓	✓(partial)	✗	✗
SimpleTOD	Research datasets	✗	✗	✗	✗
ReAct Agents	Web automation	✓	✗	✓(limited)	✗
Domain-specific apps	Single category	✗	✗	✗	✓(partial)
BookForMe	**WA Booking**	✓	✓	✓	✓

The primary research gaps addressed are:

- **Roman Urdu code-switching:** Existing DST and NLU research does not target informal WhatsApp-style Roman Urdu, presenting significant gaps in language modelling and orthographic normalisation.
- **Transactional booking guarantees:** While agent tool-use is well studied, very few works address concurrency, idempotency, and transactional safety in real-world booking systems.
- **Asynchronous conversation flow:** Most TOD frameworks assume synchronous interaction. WhatsApp requires persistent memory and session resumption across arbitrary time delays.
- **Evaluation for informal-economy SMEs:** Agent evaluation literature does not address booking systems in informal economies where reliability, low latency, and no-infrastructure vendor onboarding are essential.

BookForMe synthesises the reviewed literature into a practical LLM-driven booking agent that is novel precisely because it operates at the intersection of all four gaps listed above.

3. Software Requirement Specification (SRS)

3.1 Introduction

3.1.1 Purpose

This document defines the requirements for the **BookForMe** platform. The system's core purpose is to simplify booking of informal, local services (sports courts, salons, gaming zones, etc.) in Karachi. The conversational booking assistant is embedded directly inside the mobile application. It collects user requirements via natural language, performs clarifying questions, and books directly from the application's internal database when matching slots exist.

When the user requests slots that are not present in the application's internal database and explicitly consents to external outreach, the system triggers an **Outreach Agent** workflow. The Outreach Agent consults a maintained contact list of vendor numbers, selects candidate vendors matching the user's criteria, sends structured inquiries to those vendors, ingests their replies, and presents collected external availability options back to the user. Outreach-sourced vendors are presented as informational only (including contact details and parsed availability); the user must contact such vendors directly to complete a booking.

Additionally, all onboarded vendors can link their WhatsApp Business number to the platform's **AI Receptionist**, which then autonomously handles booking

conversations on their behalf—checking availability, locking slots via OCC, verifying payment via OCR, and notifying the vendor for final approval.

3.1.2 Scope

The BookForMe system comprises four primary components:

1. **In-App Conversational Booking Interface:** Natural-language chatbot embedded in the React Native mobile app that handles synchronous user interaction, clarifications, and direct bookings from the internal database.
2. **Vendor Dashboard:** Vendor-facing interface for onboarding, profile management, calendar viewing, booking approvals, and linking WhatsApp Business API or Google Sheets integrations.
3. **AI Receptionist (WhatsApp Agent):** Autonomous agent deployed on the vendor's WhatsApp Business number. Handles bilingual inbound booking requests end-to-end, from NLU to payment verification to vendor notification.
4. **Outreach Agent:** Upon user consent, dispatches structured inquiries to external vendor contacts, parses responses into canonical availability format, and returns informational options to the user.
5. **Core Backend Services:** Firestore database, NLU/LLM interfaces, WhatsApp Business API messaging gateway, OCR service, and Firebase Authentication.

3.2 Functional Requirements

This section describes each function/feature provided by our system. These functions are logically grouped into modules based on purpose and users.

- **Module: Customer / User (Booking & Discovery)**

- FR-CUST-01: The system shall allow users to browse and search registered vendors by service category (e.g., Futsal, Salon, Gaming Zone) and location area.
- FR-CUST-02: The system shall provide filtering options (price range, rating, amenities such as parking, WiFi, AC courts).
- FR-CUST-03: The system shall display vendor profiles including name, description, location, services offered, pricing, reviews, and ratings.
- FR-CUST-04: The system shall display any discounts or promotions offered by a vendor.
- FR-CUST-05: The system shall present users with a calendar interface showing real-time available and booked time slots.
- FR-CUST-06: The system shall allow users to select an available time slot, add it to a cart, review the booking summary, and confirm checkout.
- FR-CUST-07: The system shall allow users to upload a payment screenshot after booking confirmation.
- FR-CUST-08: The system shall confirm successful bookings to the user via the app interface, notifications page, and optionally a WhatsApp confirmation.

- **Module: Customer / User (Interaction & Feedback)**
  - FR-CUST-09: The system shall allow users to leave a star rating and text review for a vendor after a completed booking.
  - FR-CUST-10: The system shall provide an on-app AI chatbot capable of parsing natural-language queries (English / Roman Urdu) and performing cross-category filtered searches against the platform's internal database.
  - FR-CUST-11: The system shall provide a notifications page for booking status updates, friend requests, and direct messages.

- FR-CUST-12: The system shall allow users to consent to external outreach when no internal matches satisfy their request; the system shall then trigger the Outreach Agent (FR-OUT-01 – FR-OUT-10).

- **Module: Customer / User (Social & Community)**
  - FR-SOC-01: The system shall provide user registration and login (email/password and Google OAuth).
  - FR-SOC-02: The system shall allow users to create and manage a public profile (username, avatar, match history, Elo rating).
  - FR-SOC-03: The system shall allow users to search for other users on the platform.
  - FR-SOC-04: The system shall provide a friend system: send, accept, and decline friend requests.
  - FR-SOC-05: The system shall allow users to send and receive private direct messages with users on their friends list.
  - FR-SOC-06: The system shall provide a community forum where users can create, view, like, and comment on public posts.
  - FR-SOC-07: The system shall provide a mechanism for users to report other users or posts for moderation.

- **Module: Customer / User (Matchmaking & Ranking)**
  - FR-MATCH-01: The system shall provide a mechanism for users to queue for a Casual Match (no rating impact).
  - FR-MATCH-02: The system shall provide a mechanism for users to queue for a Ranked Match, matched by Elo rating within a configurable window ($\pm$200 points, expanding with wait time).
  - FR-MATCH-03: The system shall calculate and update user Elo ratings based on ranked match outcomes.

- FR-MATCH-04: The system shall display a public leaderboard of top-ranked players, filterable by sport.

- **Module: Vendor**
  - FR-VEND-01: The system shall provide secure vendor registration, login, and profile management (business name, category, location, services, pricing, operating hours, media).
  - FR-VEND-02: Vendors shall be able to link their WhatsApp Business API account to activate the AI Receptionist for inbound WhatsApp bookings.
  - FR-VEND-03: Vendors shall be able to link a Google Sheets document for automatic schedule synchronisation.
  - FR-VEND-04: Vendors shall be able to view a consolidated calendar showing all bookings from all channels (web app, WhatsApp, Sheets) in one place.
  - FR-VEND-05: The system shall provide an interface for vendors to manually add, update, search, export, and delete bookings.
  - FR-VEND-06: The system shall provide a Pending Approvals interface where vendors can review a booking (customer, service, time, payment screenshot) and approve or reject it.
  - FR-VEND-07: Vendors shall be able to view basic analytics: total bookings today, monthly revenue, occupancy rate, and recent activity.

- **Module: AI Receptionist (WhatsApp Agent)**
  - FR-AGENT-01: The system shall receive inbound WhatsApp messages via a configured Meta/Twilio webhook.
  - FR-AGENT-02: The system shall send outbound WhatsApp messages (replies, confirmations, rejection notices) via the WhatsApp Business API.
  - FR-AGENT-03: The system shall process inbound messages (English / Roman Urdu) to extract intent and slot entities (service, date, time, duration, budget, location).

- FR-AGENT-04: The system shall maintain full conversational context (dialogue state) for ongoing WhatsApp interactions, persisted in Firestore across arbitrary delays.
- FR-AGENT-05: Based on NLU output and availability, the system shall determine and execute appropriate actions (confirm availability, offer alternatives, create HOLD booking, request payment, process payment, notify vendor).
- FR-AGENT-06: The system shall implement Optimistic Concurrency Control via Firestore transactions to prevent conflicting bookings for the same time slot across all channels simultaneously.
- FR-AGENT-07: The system shall parse linked Google Sheets to identify manually entered bookings and synchronise them to the central Firestore database.
- FR-AGENT-08: After verbal confirmation of a booking, the system shall request the user to submit a payment screenshot (advance payment) and set the booking status to HOLD with a 10-minute expiry.
- FR-AGENT-09: The system shall process the uploaded payment screenshot using OCR to extract the transferred amount. If the amount does not match the required advance payment, the system shall automatically reject the screenshot and request a corrected one without notifying the vendor.
- FR-AGENT-10: Upon successful OCR verification, the system shall update the booking to PENDING_APPROVAL, attach the screenshot to the booking record, and notify the vendor on the Vendor Dashboard.
- FR-AGENT-11: Upon vendor approval, the system shall update booking status to CONFIRMED and send a confirmation message with a ticket ID to the user on WhatsApp.

- **Module: Outreach (Informational-only external discovery)**
  - FR-OUT-01: When user consents, create an Outreach Job encapsulating the user's criteria and preferences.

- FR-OUT-02: Select and rank vendor contacts from the maintained contact list based on relevance (location, service type, historical responsiveness).
- FR-OUT-03: Dispatch structured outreach inquiry messages to selected vendor contacts via the WhatsApp / SMS messaging gateway using predefined templates.
- FR-OUT-04: Receive vendor replies and normalise them into canonical availability records (start time, end time, price, hold-ability if stated).
- FR-OUT-05: Aggregate and deduplicate external availability options; compute parsing confidence for each.
- FR-OUT-06: Present each external option with vendor name, phone number, preferred contact channel, parsed availability, confidence score, and a clear disclaimer that these results are informational and require the user to contact the vendor directly.
- FR-OUT-07: Store and enforce vendor outreach consent metadata; process opt-out requests promptly.
- FR-OUT-08: Maintain an audit log of all outreach messages sent and responses received.
- FR-OUT-09: Enforce messaging-provider rate limits and implement exponential back-off and retry for failed message delivery.
- FR-OUT-10: Allow the user to cancel an ongoing Outreach Job at any time.

- **Module: Stretch Goals (Future Scope)**
  - FR-STRETCH-01: The system may provide a payment gateway (JazzCash / EasyPaisa) replacing screenshot-based payment with programmatic confirmation.
  - FR-STRETCH-02: The AI Receptionist may be extended to transcribe and understand WhatsApp voice notes via Whisper ASR.

- `FR-STRETCH-03`: The system may implement a recommendation algorithm (collaborative filtering, link prediction) to suggest vendors based on booking history and social graph.
- `FR-STRETCH-04`: The system may extend to voice-call booking via Twilio Voice with Google/Azure speech-to-text and text-to-speech.

3.2.1 Use Case Interaction (Textual Representation)

Core interactions:

- **App Customer** uses the *Mobile Application* to:
  - Register, log in, and manage a profile.
  - Search and filter vendors; view availability calendars.
  - Select a slot, confirm booking, upload payment proof, and receive confirmation.
  - Use the embedded AI chatbot for natural-language slot discovery.
  - Consent to Outreach Agent when no internal match is found.
  - Join match lobbies; post to forums; message friends; view leaderboard.
- **WhatsApp Customer** uses the *AI Receptionist* to:
  - Send a natural-language booking request (English or Roman Urdu).
  - Answer clarifying questions from the agent.
  - Receive slot options and confirm a choice.
  - Upload a payment screenshot for OCR verification.
  - Receive a booking confirmation ticket.
- **Vendor** uses the *Vendor Dashboard* to:
  - Register, manage business profile, and add services/slots.

- Link WhatsApp Business API to activate the AI Receptionist.
- Link Google Sheets for schedule synchronisation.
- View consolidated booking calendar and approve/reject pending bookings.
- View revenue and occupancy analytics.

3.3 Non-Functional Requirements

This section specifies the quality attributes of the system.

3.3.1 Performance

- `NFR-PERF-01`: Key web/app pages (vendor search, availability calendar) shall load within **30 seconds** on an average stable mobile connection.
- `NFR-PERF-02`: The AI Agent's NLU processing for a standard WhatsApp message shall complete within **60 seconds**.
- `NFR-PERF-03`: Total time from receiving a WhatsApp availability query to sending the initial agent response shall not exceed **60 seconds**.
- `NFR-PERF-04`: The WhatsApp webhook endpoint shall be capable of processing at least **4 simultaneous** incoming requests without failure.
- `NFR-OUT-PERF-01`: Outreach job creation shall be queued within **2 seconds**; first batch of outreach messages shall be scheduled within **15 seconds** of job creation, subject to provider rate limits.

3.3.2 Security and Privacy

- `NFR-SEC-01`: All user and vendor access shall be protected via Firebase Authentication (email/password and OAuth 2.0).

- `NFR-SEC-02`: All external API keys (WhatsApp, Google Sheets, Gemini) shall be stored as environment variables / Google Secret Manager secrets, never hardcoded.
- `NFR-SEC-03`: The WhatsApp webhook endpoint shall verify the integrity of all incoming requests using Meta's Verify Token.
- `NFR-SEC-04`: All data transmission between clients, the backend, and external APIs shall use HTTPS/TLS.
- `NFR-SEC-05`: Payment screenshots shall be stored in a private Firebase Storage bucket; access shall be granted only via time-limited Signed URLs to the specific vendor and admin.
- `NFR-PRIV-01`: The Outreach Agent shall only contact vendors with recorded consent; opt-out requests shall be processed within 24 hours.

3.3.3 Reliability and Robustness

- `NFR-REL-01`: The system shall target **99.0%** uptime during peak operating hours (6 PM–11 PM daily).
- `NFR-REL-02`: The AI Agent shall handle external API errors (NLU timeouts, Gemini failures) gracefully, logging errors without crashing the main service; a fallback rule-based handler shall respond to the user.
- `NFR-REL-03`: The Google Sheets synchronisation process shall be fault-tolerant, logging API and parsing errors without corrupting the main Firestore database.
- `NFR-OUT-RETRY-01`: Outreach message delivery failures shall be retried with exponential back-off (max 3 retries); all failed attempts shall be logged.

3.3.4 Scalability

- `NFR-SCAL-01`: The backend application shall support horizontal scaling via Firebase Cloud Functions (serverless) to handle increased load.

- `NFR-SCAL-02`: The Firestore schema shall support baseline functionality for at least **100 concurrent users** without significant performance degradation.
- `NFR-SCAL-03`: The system shall support onboarding and operation for at least **10 vendors** simultaneously without degradation.

3.3.5 Usability

- `NFR-USAB-01`: A new user shall be able to complete a booking in fewer than **8 primary steps** from the home screen (Search → Select Vendor → View Calendar → Select Slot → Confirm → Payment).
- `NFR-USAB-02`: All key vendor dashboard functions (calendar, approvals, connect APIs) shall be accessible within **6 taps** from the main dashboard screen.
- `NFR-USAB-03`: WhatsApp agent responses shall be clear, concise, and contextually appropriate in both English and Roman Urdu.
- `NFR-USAB-04`: The UI shall clearly distinguish between internally bookable slots (confirmed via the app) and external outreach results (requiring direct user contact).

3.3.6 Data Integrity and Backup

- `NFR-DATA-01`: Canonicalised external availability records shall include a source reference and ingestion timestamp.
- `NFR-BACK-01`: Outreach job logs and Firestore data shall be included in daily automated backups.

3.4 SRS Part 2: System Block Diagram and Higher-Level Architecture

3.4.1 System Block Diagram

Figure 3.1 presents the high-level system block diagram for the BookForMe platform. The architecture has two primary layers: a **Frontend Layer** and a **Backend / API Layer**.

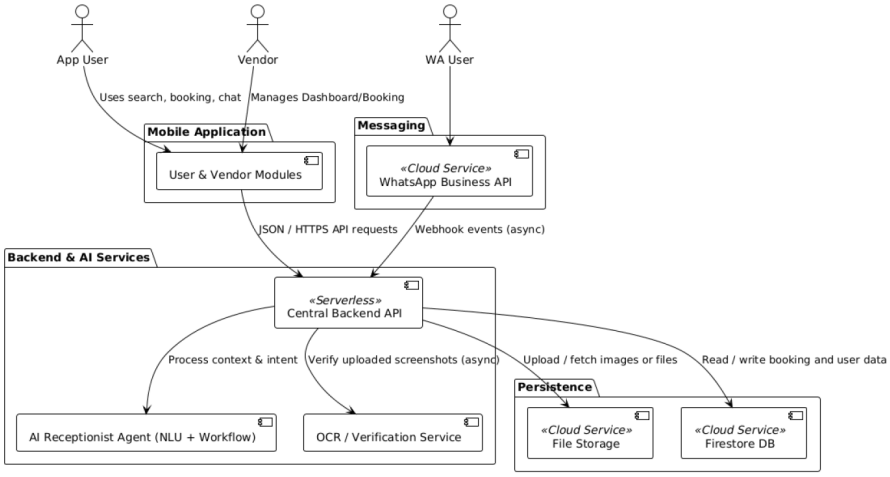


Figure 3.1: System block diagram for the BookForMe platform.

3.4.2 Description of Components

User Mobile Application (Frontend) Developed using **React Native (TypeScript)**, this is the native mobile app for customers. It allows users to browse vendors, book services, interact with the embedded AI chatbot, and access all social features (profiles, chat, forums, matchmaking). It communicates with the backend via RESTful JSON/HTTPS APIs.

Vendor Dashboard (Frontend) A secure administrative interface also built with **React Native**. It allows vendors to manage their business profile, view the

consolidated booking calendar, approve or reject pending payments, and link their external WhatsApp Business API account and Google Sheets.

Central Backend API Deployed as a **Python / FastAPI** serverless application on Render.com / Firebase Cloud Functions. It is the single orchestration layer that: processes app API calls, routes WhatsApp webhook events to the AI Agent, delegates image processing to the OCR Service, and maintains data consistency in Firestore.

AI Receptionist Agent (NLU + LangGraph Workflow) The core intelligent engine of the platform, built in Python with LangGraph. Responsible for: managing the WhatsApp webhook conversation loop, running the fine-tuned DistilBERT NLU model for intent classification and Gemini API for slot extraction, executing the LangGraph cyclic state graph (Guardrails $\rightarrow$ Classify Intent $\rightarrow$ Extract Slots $\rightarrow$ Validate State $\rightarrow$ Query Availability $\rightarrow$ Execute Booking), and enforcing OCC for all bookings.

OCR / Verification Service A Python microservice that processes payment screenshot images uploaded by WhatsApp users. It extracts the transaction amount and validates it against the required advance payment. Uses Tesseract / Google Cloud Vision OCR.

Cloud Firestore Database The central **NoSQL data repository**. Stores all persistent data: user and vendor profiles, service listings, real-time availability, booking records, conversation state for the AI Agent, payment records, and social data (forum posts, friend relationships, matchmaking lobbies).

Firebase Storage Stores payment screenshot images uploaded by users. Access is controlled via time-limited Signed URLs.

WhatsApp Business API The primary communication channel for the AI Receptionist. Receives inbound user messages as HTTP webhook events and sends replies via the Meta/Twilio WhatsApp API.

Authentication and Security Layer Firebase Authentication manages all user and vendor identities (email/password, Google OAuth). API middleware validates Firebase ID tokens and enforces RBAC (Vendor vs. Customer) on every protected route.

3.5 Project Plan

3.5.1 Objectives

The overarching project objectives are:

1. Develop a centralized booking platform connecting users and vendors across informal service domains in Karachi.
2. Implement an AI Receptionist (vendor-side) capable of processing WhatsApp booking requests using bilingual NLU.
3. Implement a Conversational Search Agent (user-side) for natural-language vendor discovery within the app.
4. Deliver integrated social features: user profiles, friends, direct messaging, forums, and matchmaking/ranking.
5. Design a scalable, secure backend integrated with Firestore and the WhatsApp Business API.
6. Deliver a seamless React Native frontend ensuring accessibility, reliability, and real-time synchronisation.

3.5.2 Team Roles and Responsibilities

- **Muhammad Taqi (AI Engineer):** Designs and implements the NLU pipeline (intent detection, slot extraction, model fine-tuning). Collaborates on AI Receptionist integration and data augmentation.

- **Muhammad Ahmad Hanif (AI Engineer):** Leads system architecture and AI Receptionist integration with Firestore. Responsible for NLU pipeline development. Oversees project roadmap, documentation, and delivery milestones.
- **Jazib Waqas (Backend Engineer):** Responsible for backend architecture, database integration, booking/vendor/authentication APIs. Leads WhatsApp API integration and OCC implementation.
- **Taha Hunaid Ali (Latency Developer):** Develops and maintains user and vendor dashboards in React Native. Works on real-time interface features (chat, leaderboard, analytics).

3.6 Wireframes

The following descriptions represent the key screens of the BookForMe mobile application. (In the submitted project, Figma-exported images replace these descriptions.)

Login / Role Selection Unified sign-in screen with a role toggle (User / Vendor), email–password fields, a "Continue with Google" OAuth button, and a "Login as Guest" option.

Customer Registration Form Three-step onboarding: basic details (first name, last name, email, mobile, password), preferences (preferred sports, location), and agreements (Terms & Privacy Policy).

Customer Home Screen Discovery hub showing: location indicator ("Karachi, Pakistan"), search bar, horizontally scrollable category tiles (Sports Courts: 156 venues, Gaming Zones: 89 venues, ...), a "Trending" section with venue cards showing name, rating, distance, and price, and an "Upcoming Bookings" widget.

Category Listing / Search Results Filterable list with left-panel filters (price/hr slider, amenity checkboxes) and right-panel venue cards (photo, name, tags, price, Book button).

Vendor Detail Screen Venue profile with photo slider, star rating, review count, distance, opening hours, description, and an interactive date–time slot picker showing available and booked slots.

Booking Confirmation / Payment Summary card (venue, date, time, service), customer details form, payment summary, and a WhatsApp confirmation checkbox.

AI Chatbot Screen Chat interface with a "BookForMe Assistant" header, example prompt chips ("Find sports courts near me", "Check my bookings"), and a free-text input.

Social Hub Tabbed view: Forum (posts, likes, comments), Matches (ranked and casual open match listings with Join button), Chats (DM list), Leaderboard.

Vendor Dashboard At-a-glance metrics (today's bookings, pending count, monthly revenue, active integrations), recent booking list with status badges (CONFIRMED, PENDING), and quick-action buttons.

Vendor Calendar Day / Week / Month toggle with colour-coded time slots (available, held, confirmed, rejected), source filter (all / WhatsApp / manual), and an Add New Slot button.

4. Software Design Specification (SDS)

This chapter provides the important design artifacts for the BookForMe project, covering the software architecture, data model, agentic pipeline, key algorithms, and workflow diagrams.

4.1 Software Design

4.1.1 Architectural Overview

BookForMe follows a **Service-Oriented Architecture (SOA)** with a strict separation between the Presentation, Business Logic, and Persistence layers. The architecture supports two distinct client types:

1. **Rich Client (Mobile App):** Connects via HTTPS / JSON. Best for power users who want to browse categories, manage a profile, and join social lobbies.
2. **Conversational Client (WhatsApp):** Connects via Webhooks to the AI Receptionist. Best for quick bookings without installing an app, using natural language in English or Roman Urdu.

Both clients converge on a single backend and Firestore database—the **Single Source of Truth**—ensuring that a booking made via WhatsApp is immediately reflected in the mobile app and on the vendor dashboard. The Central Backend API

is the sole orchestrator; it delegates to the AI Agent and OCR Service for specialised logic. Dependencies are strictly unidirectional: Clients depend on Backend; Backend depends on Database.

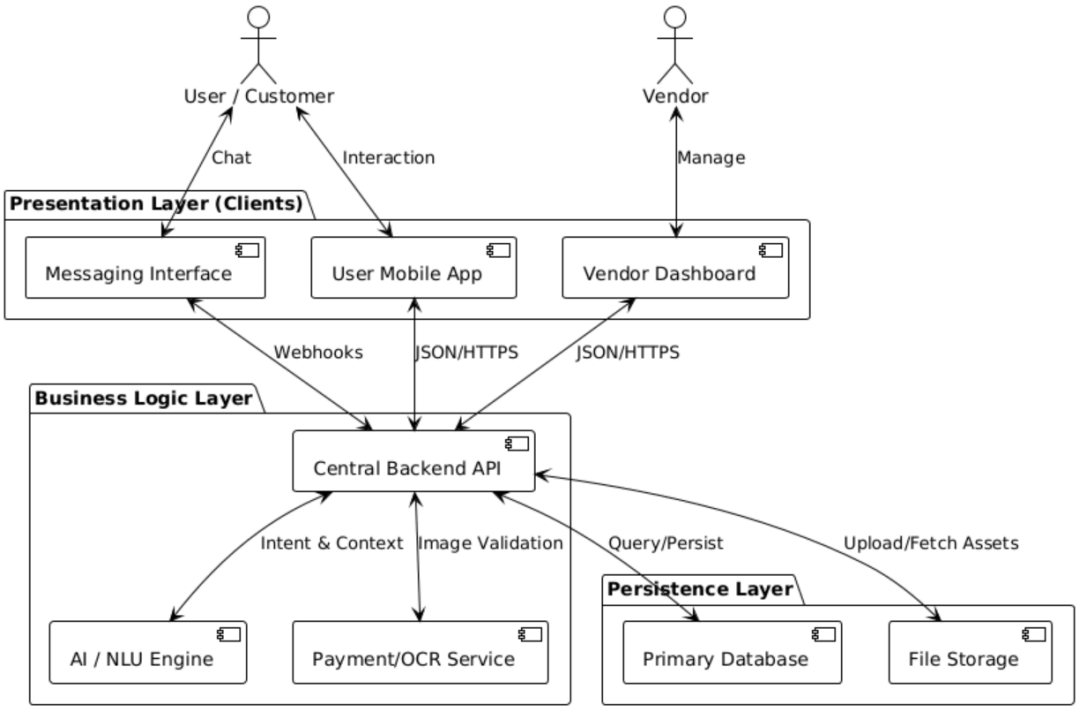


Figure 4.1: Primary system architecture of BookForMe.

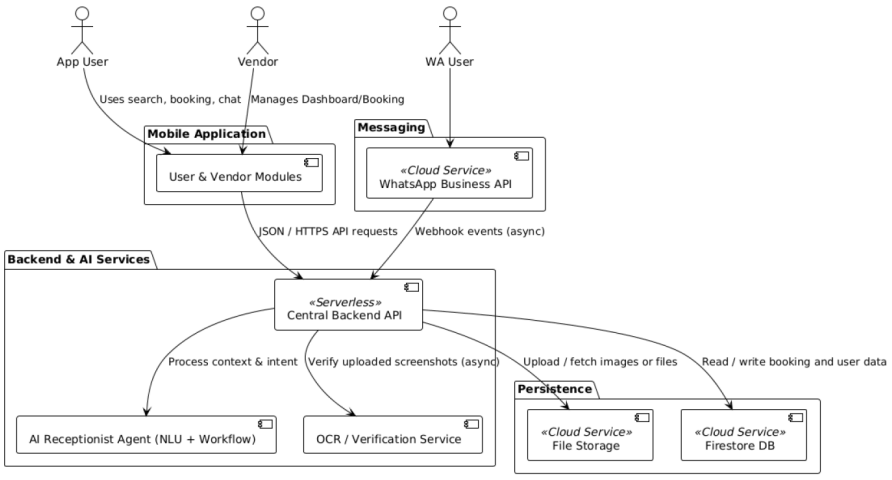


Figure 4.2: System overview showing major platform components.

4.1.2 Architectural Design Decisions

Neuro-Symbolic AI Architecture The LLM (Gemini) interprets natural language (creative / linguistic), but all database mutations are executed by deterministic LangGraph code nodes (symbolic / strict logic). This prevents the agent from "hallucinating" bookings or bypassing payment requirements. It also provides a natural security boundary against prompt-injection attacks.

Cyclic State Graph (LangGraph) Unlike linear DAG-based pipelines, the LangGraph cyclic state graph can loop back—e.g., returning to "Classify Intent" after receiving a corrected payment screenshot. This is essential for WhatsApp's asynchronous, multi-turn nature.

Reference-Based NoSQL Schema Firestore's document model is used with a Reference-Based strategy: entities store document references (IDs) rather than embedding full sub-documents. This balances NoSQL flexibility with structured relational integrity and avoids document-size limits.

Strict Social / Transactional Separation The Firestore schema separates the **Social Ecosystem** (user reputation, matchmaking, forums) from the **Trans-**

actional Domain (vendors, services, bookings, payments). High-volume social interactions cannot degrade the performance or integrity of critical financial data.

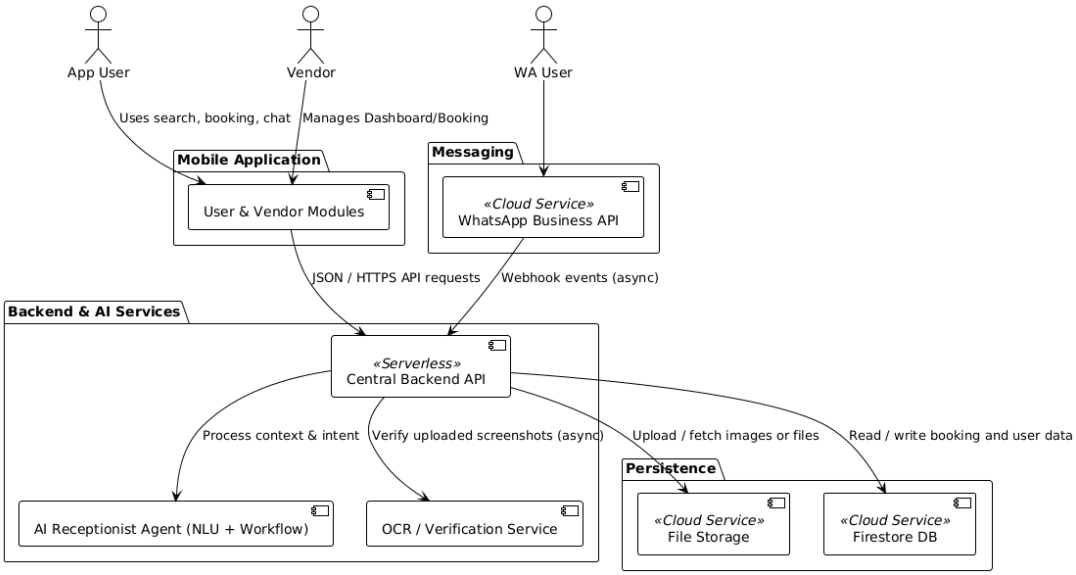


Figure 4.3: Software Architecture Diagram (SAD) for BookForMe.

4.2 Data Design

4.2.1 Database Strategy

Data persistence is handled by **Google Cloud Firestore**, a serverless NoSQL document database. The schema is divided into two domains:

- **User & Social Domain:** Manages user engagement (reputation, matchmaking, friends, forums, DMs) without affecting financial data integrity.
- **Core Booking Domain:** Manages all business logic (vendors, services, bookings, payments) with strong consistency guarantees via Firestore transactions.

4.2.2 Entity Descriptions

Table 4.1 describes the primary data entities and their roles.

Table 4.1: Domain entity descriptions.

Entity	Key Attributes	Description
`Users`	`user_id`, `auth_uid`, `elo_rating`, `skill_profile`	Central anchor bridging social features (Elo ratings) and commercial features (booking history).
`Vendors`	`vendor_id`, `location_geo`, `category`, `owner_ref`	Business entity defined independently of services to support scalable inventory management.
`Services`	`service_id`, `vendor_ref`, `price_per_slot`, `type`	The sellable unit (e.g., a specific Futsal Court or Gaming PC).
`Bookings`	`booking_id`, `user_ref`, `service_ref`, `hold_expires_at`, `status`, `payment_ref`	Transactional record. Includes `hold_expires_at` timestamp for OCC slot locking. Status: {HOLD, PENDING_APPROVAL, CONFIRMED, REJECTED}.
`Payments`	`transaction_id`, `screenshot_url`, `ocr_verified_amount`, `status`	Separated from `Bookings` for auditability. Stores raw screenshot and OCR-validated amount. Status: {PENDING, VERIFIED, REJECTED}.
`Match_Lobbies`	`match_id`, `host_ref`, `participants`, `mode`, `avg_elo`, `status`	Manages matchmaking queues for Ranked and Casual lobbies.
`Forum_Posts`	`post_id`, `author_ref`, `content`, `timestamp`, `comments` (sub-collection)	Community posts driving social engagement.
`Direct_Messages`	`chat_id`, `participants`, `last_message`, `last_timestamp`	Peer-to-peer chat between users on the friends list.

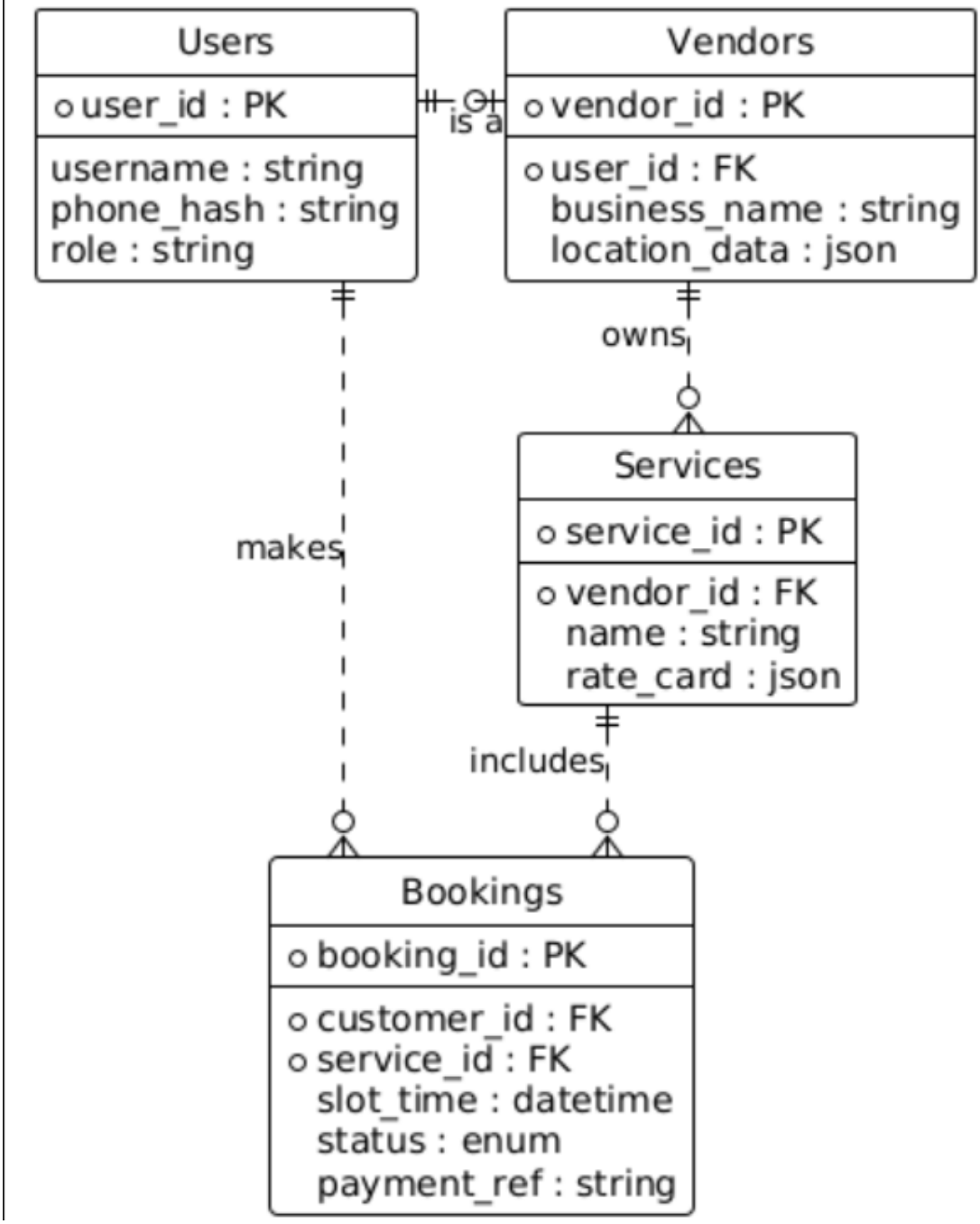


Figure 4.4: Entity Relationship Diagram (ERD) for the core data model.

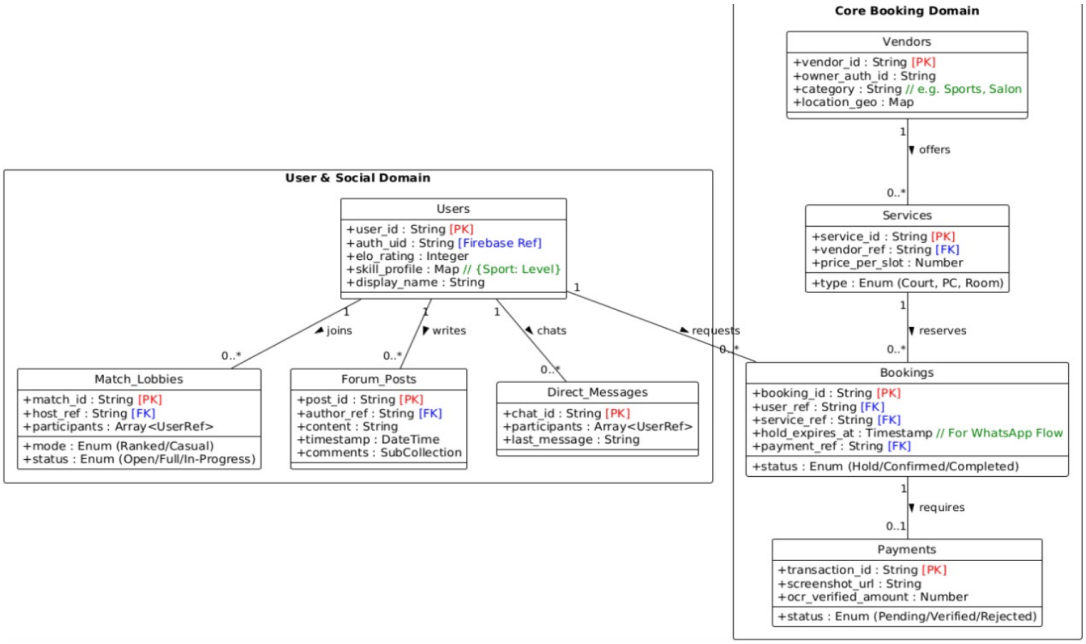


Figure 4.5: Detailed data model used for Firestore document structure.

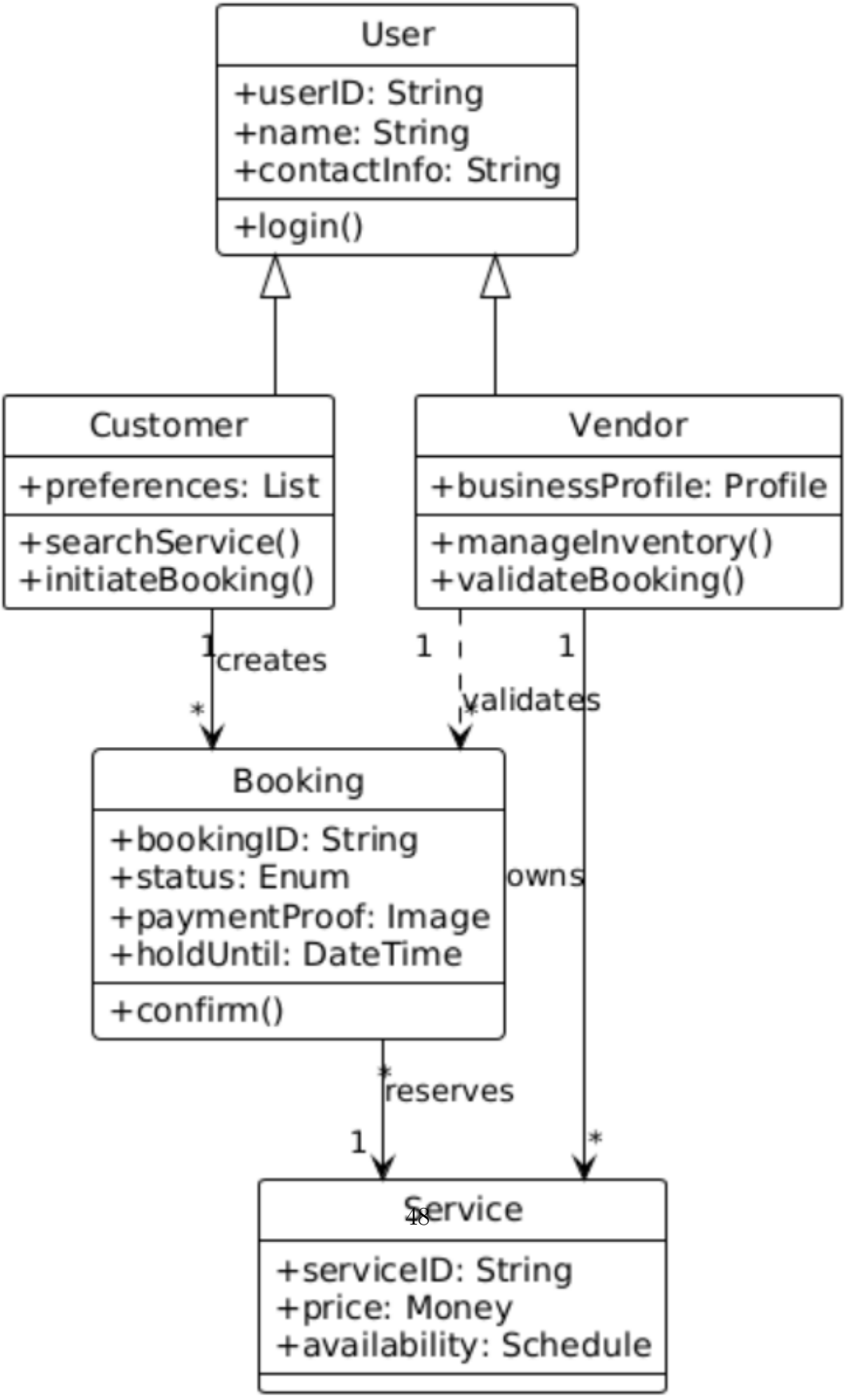


Figure 4.6: Class diagram for major domain objects and relationships.

4.2.3 Booking Status Lifecycle

Figure 4.7 shows the state machine for a booking record.

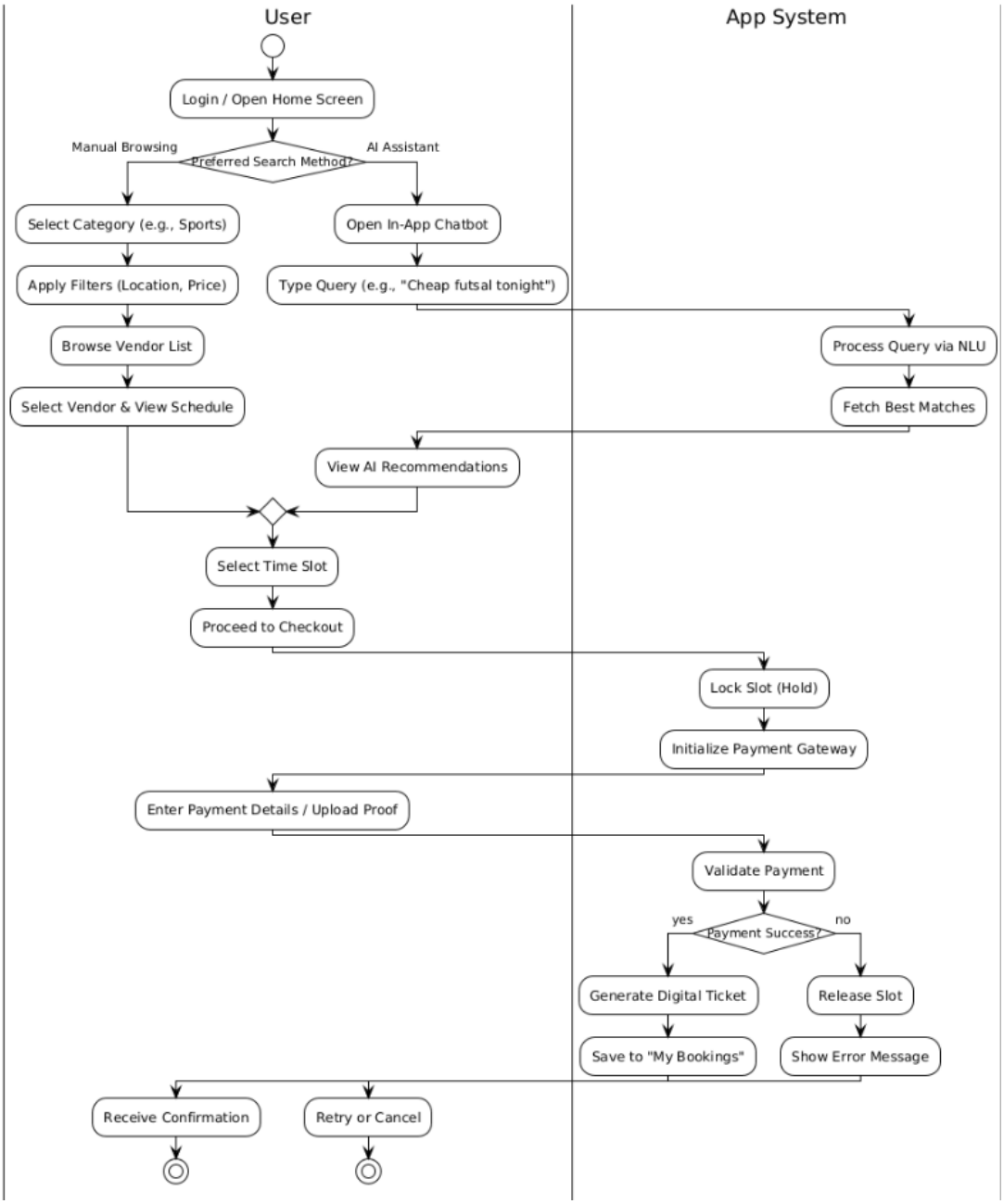


Figure 4.7: Booking record state lifecycle.

4.3 Technical Details

4.3.1 Key Algorithms

Algorithm 1: Intent Parsing and Action Routing

Purpose: Translate unstructured natural language into deterministic database actions without hallucination.

Input: Raw WhatsApp message string + serialised user session state.

Logic:

1. The fine-tuned DistilBERT model classifies the user's message into one of five intent classes: `inquiry`, `info`, `transaction_confirm`, `greeting`, `unknown`.
2. The Gemini API (fallback for complex or ambiguous cases) extracts slot entities: service type, date, time, duration, budget, location.
3. The classified intent routes to a specific LangGraph node which executes rigid Python code (e.g., a Firestore query) rather than allowing the LLM to generate queries itself.
4. If required slots are missing, the Validate State node generates a targeted clarification question and loops back to Classify Intent.

Output: Validated JSON payload routed to the appropriate action node, or a clarification prompt sent back to the user.

Algorithm 2: Optimistic Concurrency Control (OCC)

Purpose: Prevent race conditions (double-bookings) in the distributed Firestore environment.

Input: `service_id`, `time_slot`, `user_id`.

Logic:

1. **Read:** Fetch the current document for the requested time slot.

2. **Verify:** Check that `status == "AVAILABLE"` and that no unexpired HOLD exists (`hold_expires_at > now()`).

3. **Write (Atomic):** Within a Firestore transaction, update `status` to `"HOLD"` and set `hold_expires_at` to `now() + 10 minutes`. This atomic write automatically fails if the underlying document was modified since the Read step.

4. **On conflict:** Query the next three available slots from the same vendor and offer them to the user as alternatives.

Output: Transaction success with `booking_id`, or a conflict error triggering the alternative-slot flow.

Algorithm 3: Elo-Based Matchmaking

Purpose: Pair users of similar skill level for Ranked sport matches.

Input: User's current Elo rating R_u, preferred sport, accumulated wait time t.

Logic:

1. Compute the search window: $\Delta = 200 + 50 \cdot \lfloor t/60 \rfloor$ (expands by 50 points for every additional minute waiting).

2. Query `Match_Lobbies` collection for open lobbies of the same sport where $|R_u - \text{avg\_elo}| \leq \Delta$ and `status == "Open"`.

3. If a lobby is found, add the user to `participants` and check if the lobby is now full; if full, transition to `"In-Progress"`.

4. If no lobby is found, create a new lobby with `avg_elo = R_u`.

Output: `lobby_id` for an existing or newly created lobby, or a WAIT signal if the user is the first in a new lobby.

Algorithm 4: Elo Rating Update

After a ranked match completes, both players' ratings are updated using the standard Elo formula:

$$R' = R + K \cdot (S - E) \tag{4.1}$$

where R is the current rating, $K = 32$ is the development coefficient, $S \in \{0, 0.5, 1\}$ is the actual score (loss/draw/win), and:

$$E = \frac{1}{1 + 10^{(R_{\text{opp}} - R)/400}} \tag{4.2}$$

is the expected score against an opponent with rating R_{opp} [8].

4.3.2 The LangGraph Agent State Machine

Figure 4.8 illustrates the full LangGraph node graph for the AI Receptionist.

Missing figure file: `images/sds/langgraph-state-machine.png`
LangGraph state machine placeholder

Figure 4.8: LangGraph cyclic state graph for the AI Receptionist.

4.3.3 Workflow Visualisations

Workflow 1: The Agentic (WhatsApp) Booking Experience

The agentic booking pipeline proceeds through four phases:

Phase 0 — Natural Language Negotiation The agent maintains a conversational loop, extracting booking intent and all required slot entities (service, date, time, budget, location) before querying the database. Incomplete requests trigger clarification prompts in the user's preferred language.

Phase 1 — Concurrency Lock Once intent is finalised, an atomic Firestore transaction creates a HOLD booking with a 10-minute `hold_expires_at`. Concurrent conflicting requests fail at this transaction and receive alternative slot suggestions.

Phase 2 — Intelligent Payment Verification The agent requests a payment screenshot (advance payment = 50% of total). The OCR service extracts the amount. If it does not match, the system auto-rejects and re-prompts without alerting the vendor. This loop continues until a valid screenshot is received.

Phase 3 — Human-in-the-Loop Vendor Approval Only validated payment records are forwarded to the Vendor Dashboard as push notifications. The vendor reviews the booking and payment proof and approves or rejects. Approval sends the user a confirmation ticket; rejection releases the held slot.

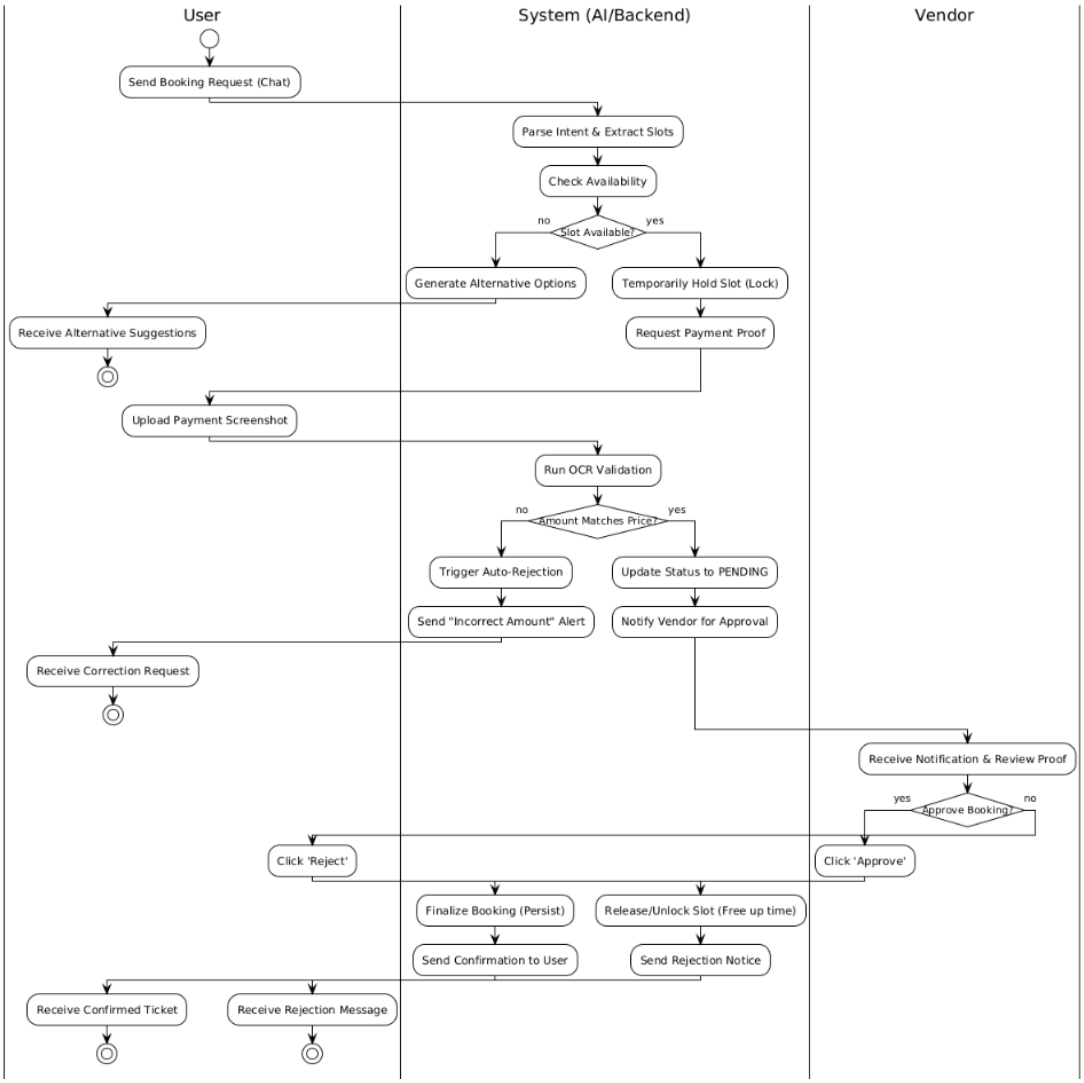
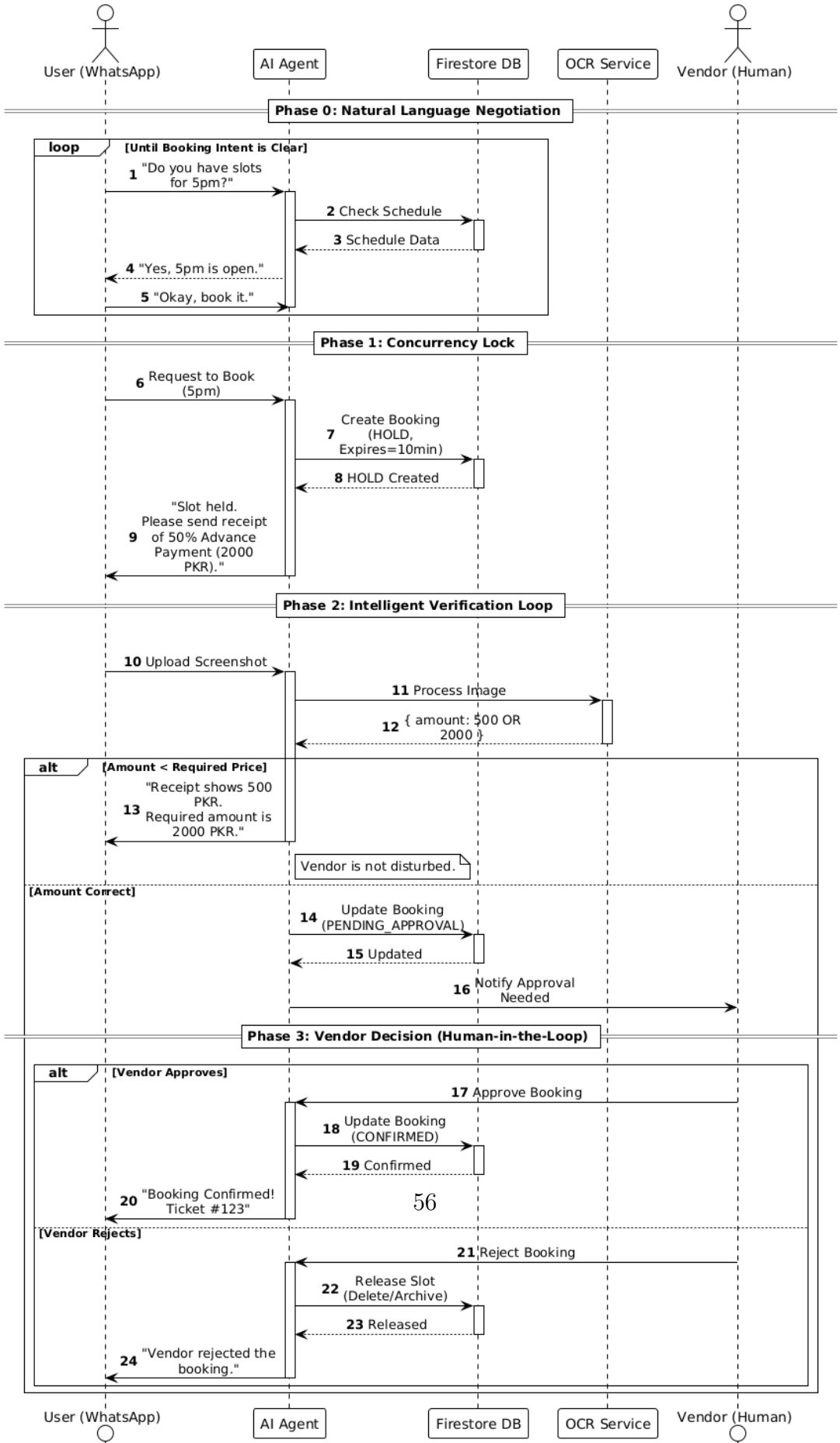
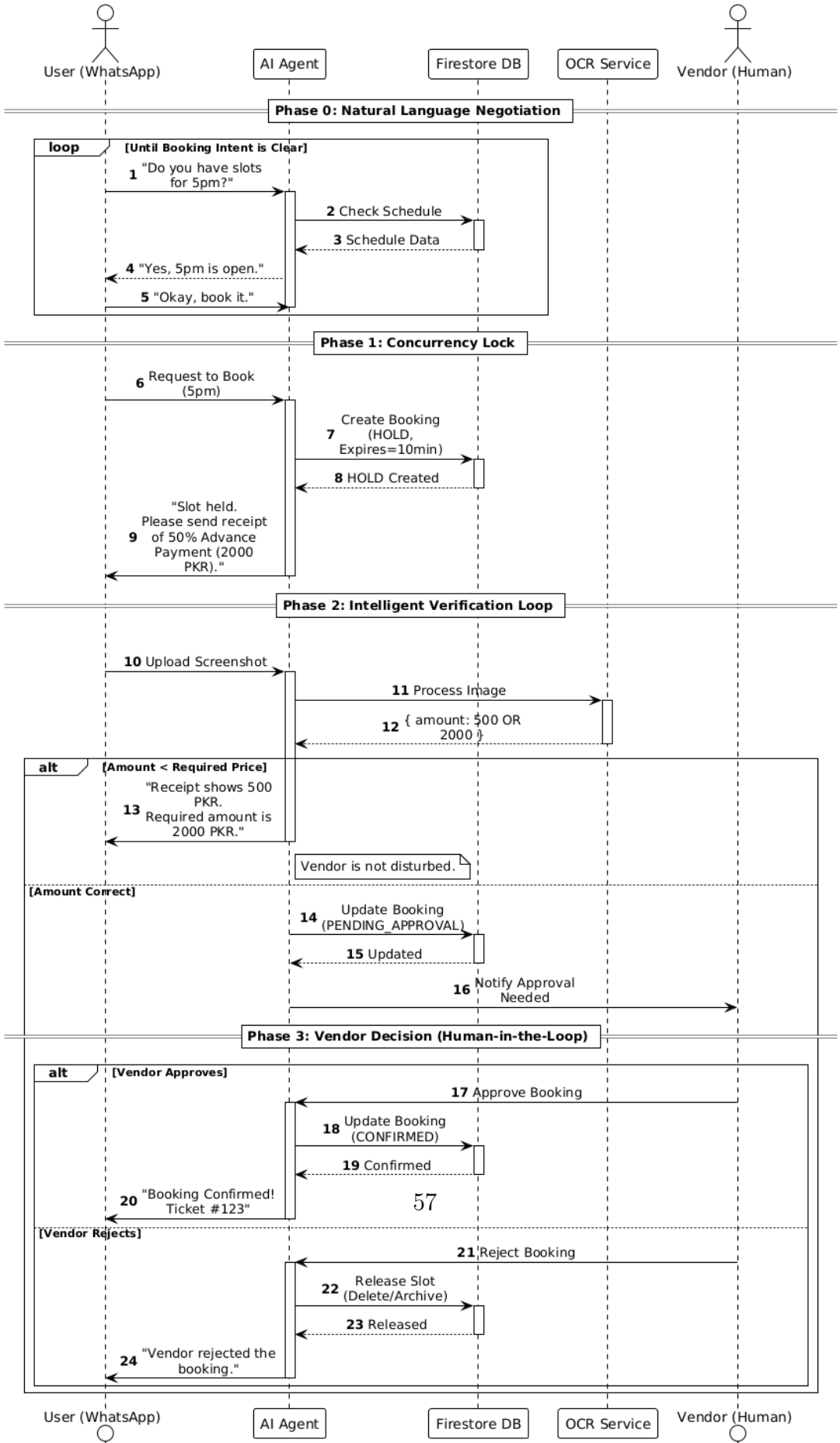


Figure 4.9: Activity diagram of the WhatsApp booking flow.


User (WhatsApp)
AI Agent
Firestore DB
OCR Service
Vendor (Human)
Phase 0: Natural Language Negotiation
loop
[Until Booking Intent is Clear]
1 "Do you have slots for 5pm?"
2 Check Schedule
3 Schedule Data
4 "Yes, 5pm is open."
5 "Okay, book it."
Phase 1: Concurrency Lock
6 Request to Book (5pm)
7 Create Booking (HOLD, Expires=10min)
8 HOLD Created
9 "Slot held. Please send receipt of 50% Advance Payment (2000 PKR)."
Phase 2: Intelligent Verification Loop
10 Upload Screenshot
11 Process Image
12 { amount: 500 OR 2000 }
alt
[Amount < Required Price]
13 "Receipt shows 500 PKR. Required amount is 2000 PKR."
Vendor is not disturbed.
[Amount Correct]
14 Update Booking (PENDING_APPROVAL)
15 Updated
16 Notify Approval Needed
Phase 3: Vendor Decision (Human-in-the-Loop)
alt
[Vendor Approves]
17 Approve Booking
18 Update Booking (CONFIRMED)
19 Confirmed
20 "Booking Confirmed! Ticket #123"
[Vendor Rejects]
21 Reject Booking
22 Release Slot (Delete/Archive)
23 Released
24 "Vendor rejected the booking."
User (WhatsApp)
AI Agent
Firestore DB
OCR Service
Vendor (Human)


User (WhatsApp)
AI Agent
Firestore DB
OCR Service
Vendor (Human)
Phase 0: Natural Language Negotiation
loop
[Until Booking Intent is Clear]
1 "Do you have slots for 5pm?"
2 Check Schedule
3 Schedule Data
4 "Yes, 5pm is open."
5 "Okay, book it."
Phase 1: Concurrency Lock
6 Request to Book (5pm)
7 Create Booking (HOLD, Expires=10min)
8 HOLD Created
9 "Slot held. Please send receipt of 50% Advance Payment (2000 PKR)."
Phase 2: Intelligent Verification Loop
10 Upload Screenshot
11 Process Image
12 { amount: 500 OR 2000 }
alt
[Amount < Required Price]
13 "Receipt shows 500 PKR. Required amount is 2000 PKR."
Vendor is not disturbed.
[Amount Correct]
14 Update Booking (PENDING_APPROVAL)
15 Updated
16 Notify Approval Needed
Phase 3: Vendor Decision (Human-in-the-Loop)
alt
[Vendor Approves]
17 Approve Booking
18 Update Booking (CONFIRMED)
19 Confirmed
20 "Booking Confirmed! Ticket #123"
[Vendor Rejects]
21 Reject Booking
22 Release Slot (Delete/Archive)
23 Released
24 "Vendor rejected the booking."
User (WhatsApp)
AI Agent
Firestore DB
OCR Service
Vendor (Human)

Workflow 2: The App User Experience

A key design feature is the **Dual-Search Capability**: users can either (a) browse vendors manually using category and filter screens, or (b) open the embedded AI Chatbot and type a natural-language query (e.g., "Cheap futsal tonight in DHA"). The chatbot uses the same NLU pipeline as the WhatsApp agent to process queries and returns matching vendor cards. Both paths converge on the same slot-selection, checkout, and payment-upload flow.

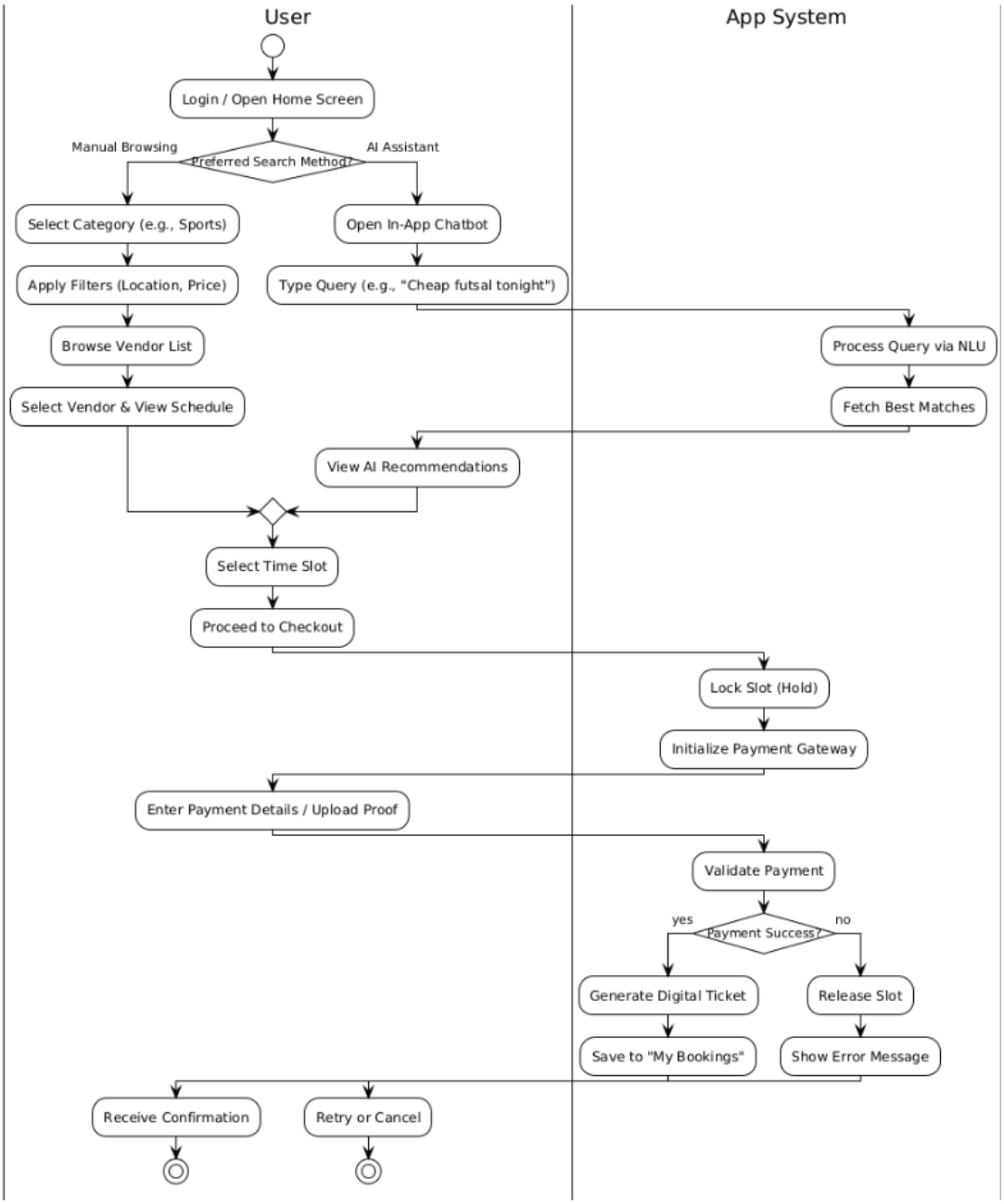


Figure 4.12: Activity diagram for the in-app booking process.

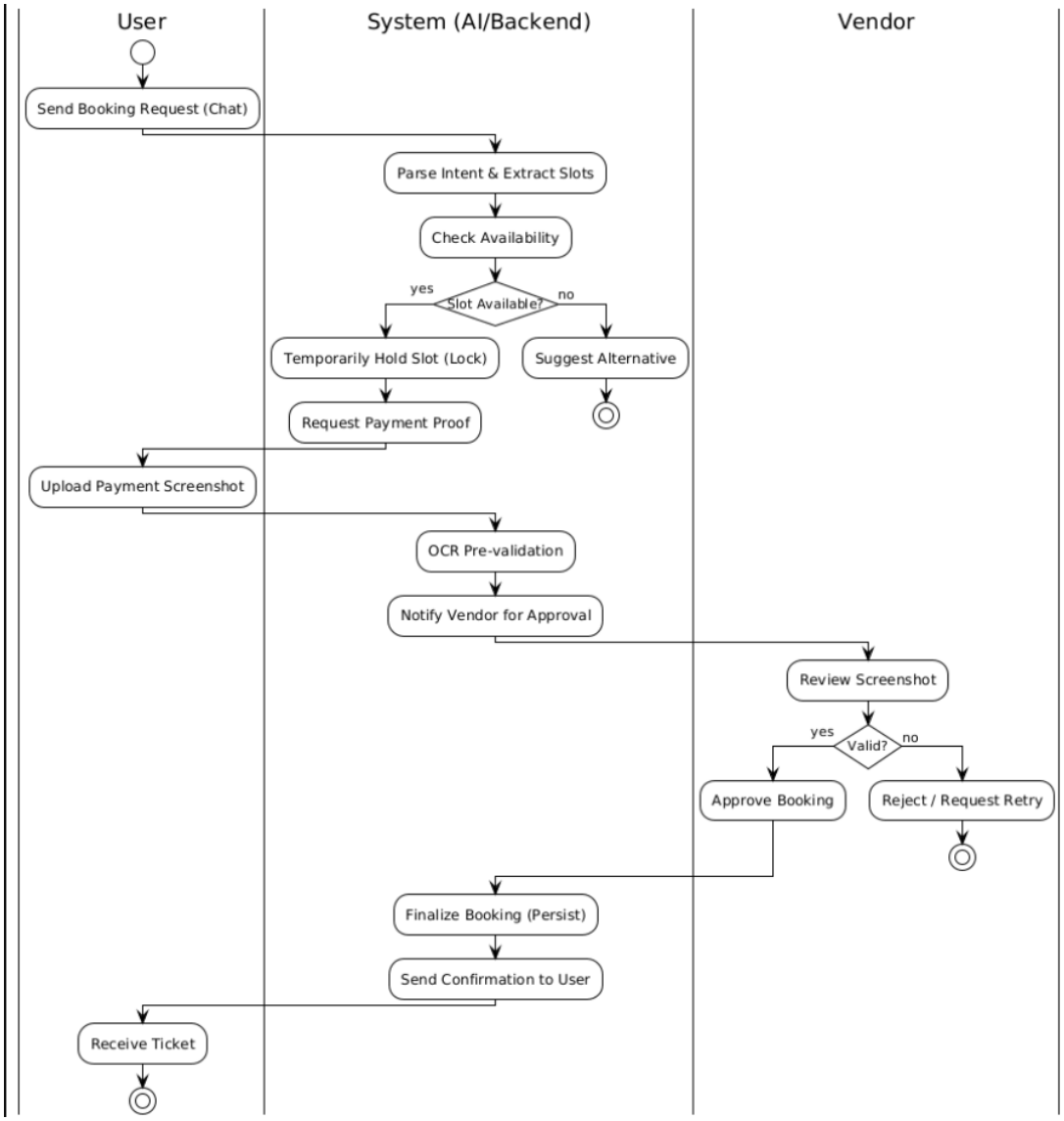


Figure 4.13: General activity diagram of the booking lifecycle.

Workflow 3: The Vendor Management Experience

Vendors interact exclusively via the Vendor Dashboard. Upon receiving a push notification for a pending booking, the vendor views: customer name, service type, requested date/time, advance payment amount, and the OCR-validated payment

screenshot. The vendor then taps Approve or Reject. Approval triggers a Firestore write to CONFIRMED and dispatches a WhatsApp confirmation to the customer. Rejection triggers a cascading slot release (AVAILABLE) and a WhatsApp rejection notice to the customer, ensuring the system never deadlocks.

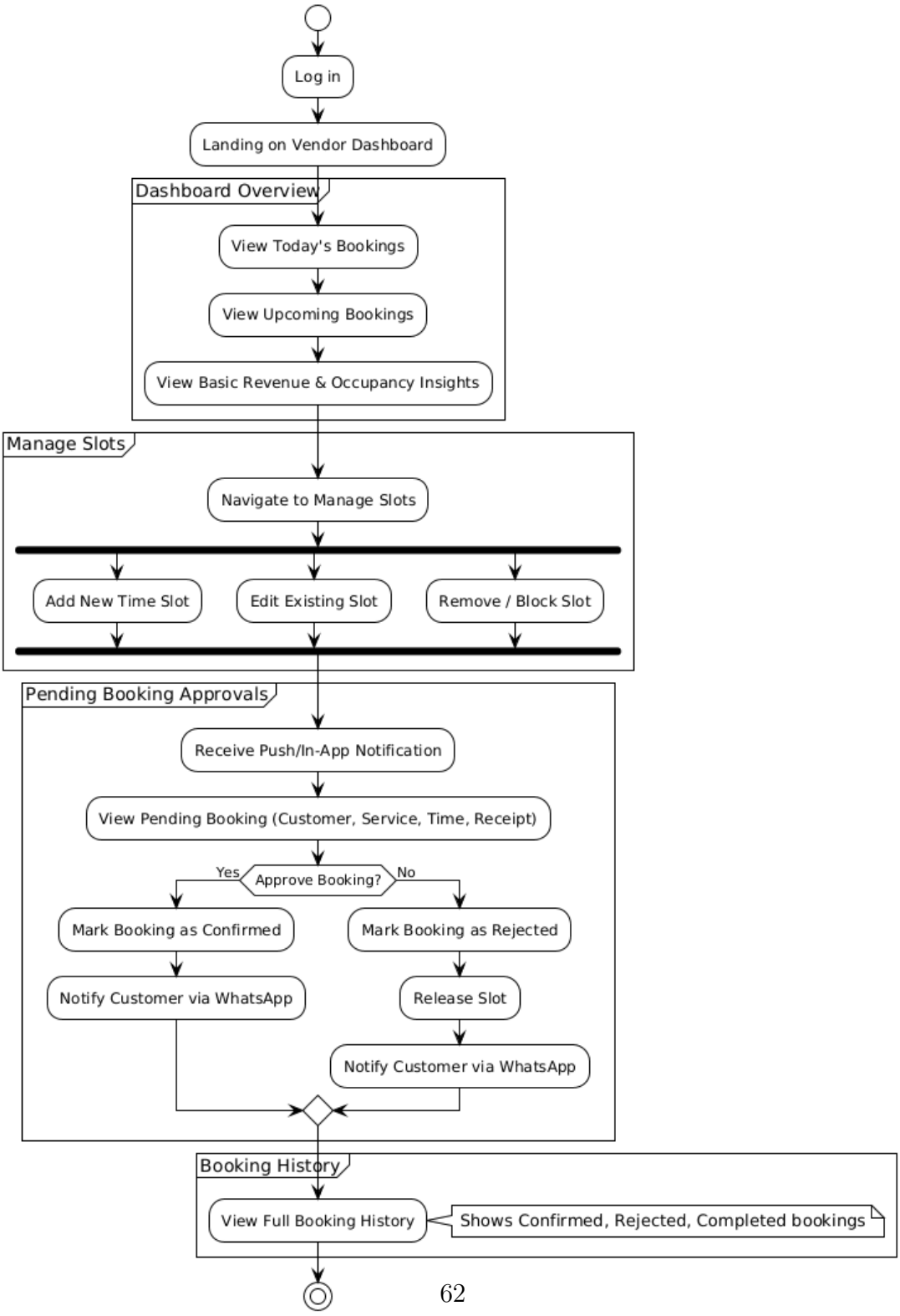


Figure 4.14: Vendor dashboard activity diagram for booking approval handling.

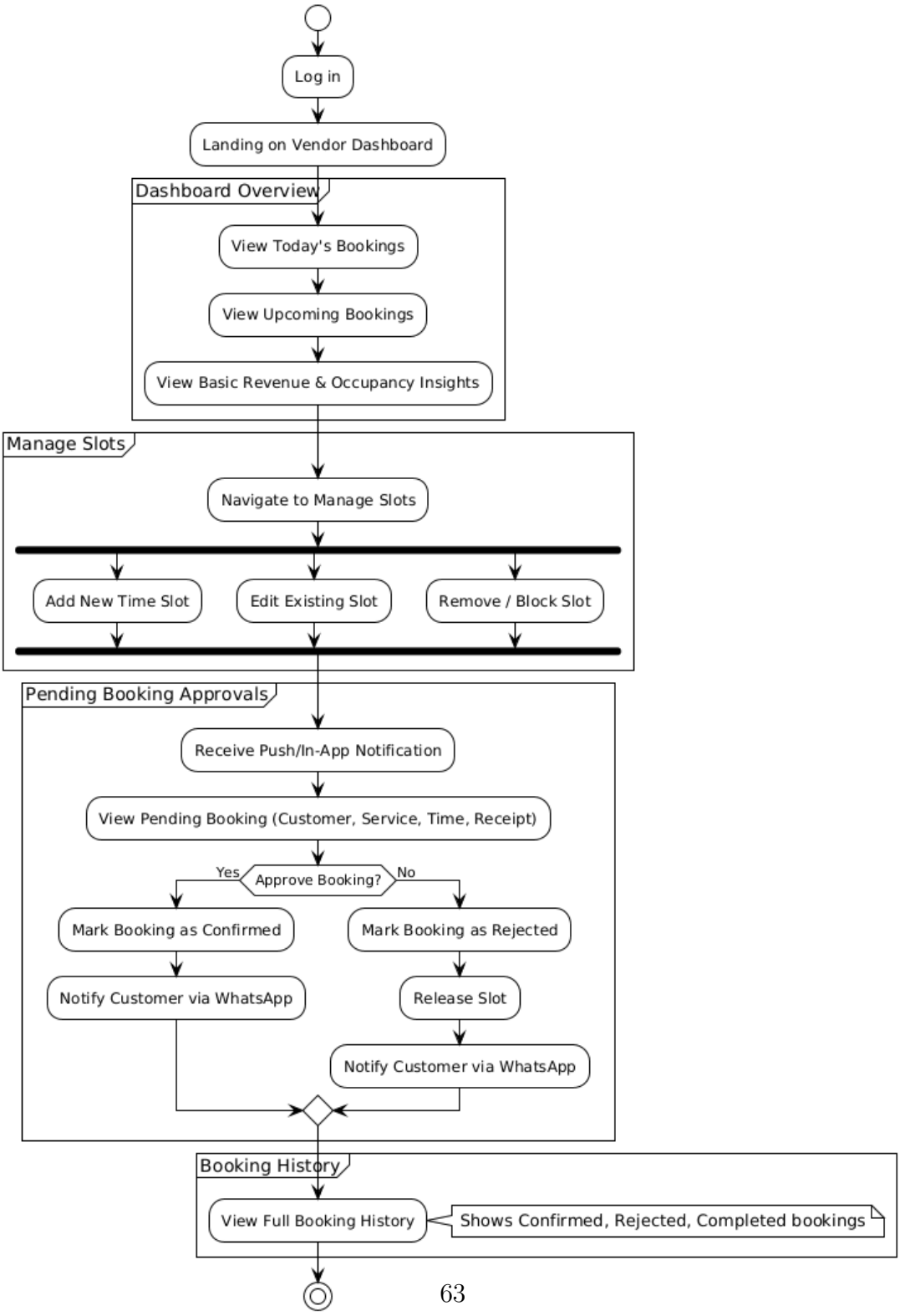


Figure 4.15: Vendor activity diagram variant for exception-handling paths.

4.3.4 Security Implementation

Authentication Firebase Auth binds the secure `auth_uid` token to all application data. The API middleware validates ID tokens on every request.

Role-Based Access Control (RBAC) Custom API middleware validates user claims (Vendor vs. Customer) before granting write access to critical Firestore collections (Services, Bookings, Approvals).

AI Guardrails (Prompt Injection Prevention) The Neuro-Symbolic architecture means the LLM cannot issue database commands directly. It classifies intent and extracts entities; all writes go through deterministic Python code nodes. Malicious prompt injections attempting to bypass payment logic are structurally impossible.

Data Privacy Payment screenshots are stored in a private Firebase Storage bucket. Time-limited Signed URLs are generated per-request and scoped to the specific vendor and admin only. Screenshots are never publicly accessible.

5. Methodology, Experiments and Results

This chapter describes the methodology used to build and evaluate the BookForMe platform, covering dataset construction, model training and selection, end-to-end agent evaluation, and non-functional benchmark results.

5.1 Dataset Construction

5.1.1 Data Collection

Real Vendor Conversations With informed consent, we collected WhatsApp booking conversation logs from five futsal and padel court operators in Karachi (DHA, Gulshan, Clifton areas). Each message was assigned an intent label and relevant slot annotations. This source provided approximately 120 utterances covering realistic vocabulary and code-switching patterns.

Manual Annotation Team members wrote and annotated booking queries in both English and Roman Urdu, targeting underrepresented intent classes (`transaction_confirm`, `unknown`). This contributed approximately 180 utterances.

LLM-Driven Synthetic Augmentation Following SynthDST [10] and An et al. [1], we used Claude Sonnet 4.5 to generate paraphrases of existing utterances in both languages, and to synthesise entirely new dialogues for rare

classes. Augmentation parameters included: back-translation (English → Urdu → English), lexical substitution (synonymous time expressions), and phonetic Roman Urdu rewriting (*kal* → *kull*, *raat* → *raat ko*, etc.). This contributed approximately 360 utterances.

5.1.2 Dataset Statistics

The full corpus comprises **694 annotated utterances** across five intent classes. Table 5.1 shows the intent distribution over the complete dataset (before train-only augmentation).

Table 5.1: Intent distribution in the full BookForMe bilingual corpus (694 samples).

Intent Class	**Count**	**% of Total**
`inquiry`	277	39.9%
`info`	171	24.6%
`transaction_confirm`	147	21.2%
`unknown`	62	8.9%
`greeting`	37	5.3%
Total	**694**	**100%**

`inquiry`: user asking to check or book a slot. `info`: user asking about price, amenities, or availability in general. `transaction_confirm`: user confirming a payment or referencing a transaction. `unknown`: out-of-scope or unintelligible messages. `greeting`: salutations and conversation openers (e.g., "hi", "salam").

The dataset was split into **485** training, **104** validation, and **105** test samples (all original, stratified; minimum 5 samples per class in the test set). Data augmentation was applied **only to the training split**, adding **31** synthetic samples by increasing `greeting` from 26 to 50 and `unknown` from 43 to 50, yielding a final training set of **516** samples.

5.1.3 Annotation Schema

Each utterance was labelled with:

- `intent`: one of the five classes above.
- `language`: `EN` (English), `UR` (Roman Urdu), or `MIXED` (code-switched).
- `slots`: a JSON dictionary of extracted entities (service, date, time, budget, location, duration).

5.2 NLU Model Training and Selection

5.2.1 Training Configuration

All six models were fine-tuned on the same dataset split with identical hyperparameters:

Table 5.2: Hyperparameters used for all fine-tuning experiments.

Parameter	**Value**
Max sequence length	128 tokens
Batch size	16
Learning rate	2e-5
Number of epochs	10
Weight decay	0.01
Warmup ratio	0.1
Evaluation steps	50
Loss function	Focal loss ($\gamma = 2.0$) for class imbalance
Min samples per class	50 (augmentation applied to minority classes)

Focal loss [1] was applied to address the class imbalance down-weighting easy majority examples and focusing training on the harder minority class boundaries.

5.2.2 Model Comparison

Table 5.3 summarises all six models evaluated on the held-out test set.

Table 5.3: Overall NLU model comparison (5-class intent classification).

Model	Notes	Accuracy	Macro-F1	Wtd. F1
bert-base-uncased	English-only pretrain; strong baseline	84.8%	87.8%	84.8%
distilbert-base-multilingual-cased	Compact multilingual; slightly slower	84.8%	83.1%	84.9%
distilbert-base-uncased	**Best overall; fastest inference (≈47 ms)**	**90.5%**	**91.7%**	**90.5%**
XLM-RoBERTa-base	Underfit on small dataset; high variance	70.5%	72.3%	69.7%
bert-base-multilingual-cased	Strong multilingual; 2nd best overall	89.5%	90.5%	89.5%
bert-base-multilingual-uncased	Slight drop vs. cased variant	84.8%	88.1%	84.7%

5.2.3 Selected Model: DistilBERT

DistilBERT (`distilbert-base-uncased`) was selected for production deployment based on:

1. **Best accuracy** (90.5%) and **macro-F1** (91.7%) across all six models tested.
2. **Fastest inference**: ≈47 ms per query on CPU, versus ≈120 ms for bert-base-multilingual-cased. This is critical given the 60-second total WhatsApp response budget (`NFR-PERF-03`).
3. **Smallest memory footprint** among competitive models (66M parameters), enabling cost-effective deployment on free-tier cloud infrastructure.

5.2.4 Per-Class Analysis (DistilBERT)

Figure 5.1 reports per-class precision, recall, and F1 for the selected DistilBERT model.

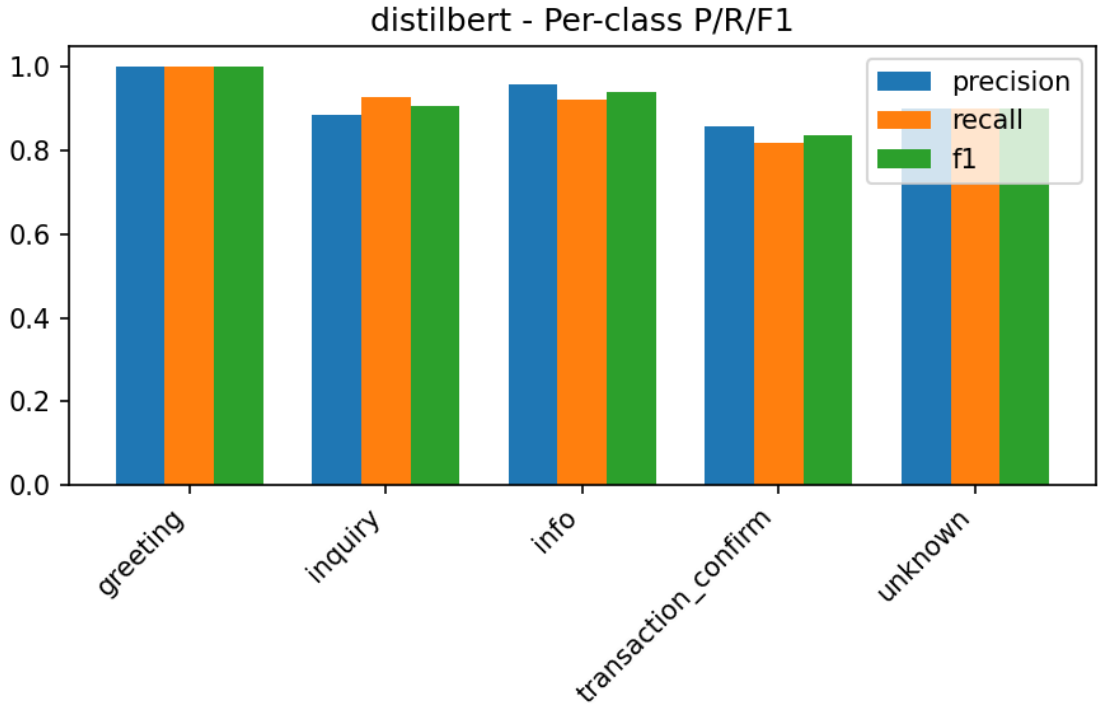


Figure 5.1: DistilBERT per-class classification report on the held-out test set

Notable finding: `transaction_confirm` shows the lowest recall (0.82), with 18% of its samples misclassified as `inquiry`. This is expected: payment-related queries ("I've sent the amount") share vocabulary with availability inquiries ("I want to confirm..."). The agent's dialogue state validation layer mitigates this—the agent checks conversation state before executing any booking action, catching misclassified payment messages.

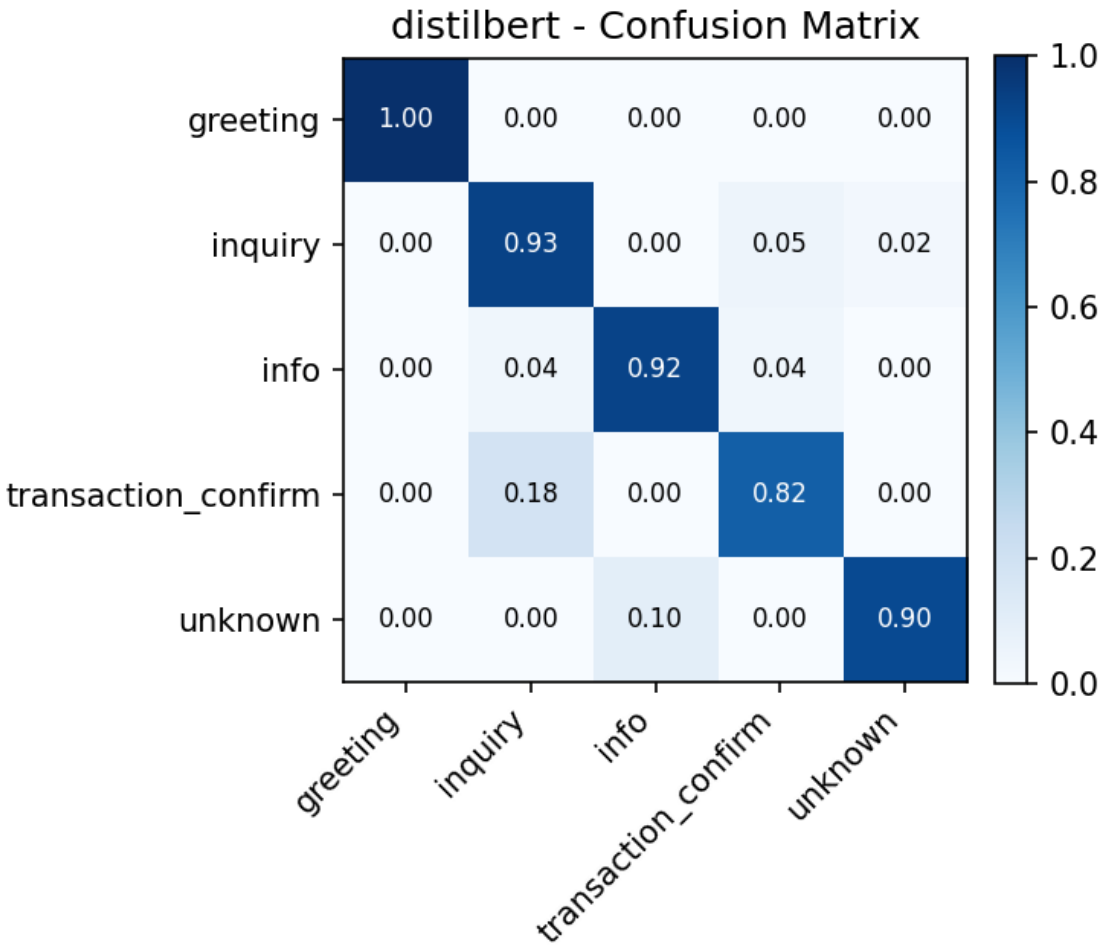


Figure 5.2: Confusion matrix for DistilBERT

5.3 Agent Evaluation

5.3.1 Test Case Methodology

The AI Receptionist was tested on the live deployment against a comprehensive test case document covering seven categories of agent behaviour. Key results are showcased below.

5.3.2 Test Case Results

Table 5.4 summarises the results of the key test scenarios.

Table 5.4: AI Receptionist agent test case results.

Test ID	Scenario	Expected / Actual Behaviour	Result
TC-01	Random out-of-scope message ("I want pineapple juice")	Agent declines politely and redirects to booking.	✓
TC-02	Requested slot unavailable (e.g., Ace Padel 10 PM tonight)	Agent offers three nearest alternative slots from same or nearby vendor.	✓
TC-03	Mid-conversation context switch in Roman Urdu ("padel kal")	Agent correctly parses date reference as tomorrow; time disambiguated via follow-up question.	✓
TC-04	OCR: payment screenshot with incorrect amount (Rs 500 vs required Rs 2000)	Agent auto-rejects, informs user of discrepancy, requests corrected screenshot. Vendor is not notified.	✓
TC-05	OCR: unreadable / low-quality image	Agent requests a clear screenshot; booking hold maintained.	✓
TC-06	Successful end-to-end booking flow (English)	Booking confirmed, ticket ID `APD-14` issued, Rs 2000 verified.	✓
TC-07	Successful end-to-end booking flow (Roman Urdu)	Agent handles "raat" (night) time reference; booking confirmed in Urdu.	✓
TC-08	Concurrent booking (OCC): two users request same slot simultaneously	Only one booking succeeds; second user receives conflict error and alternatives.	✓
TC-09	Hold expiry: user does not complete payment within 10 minutes	Booking status reverts to AVAILABLE; user notified that hold has expired.	✓
TC-10	Vendor rejects booking after payment verification	Slot released to AVAILABLE; user notified of rejection via WhatsApp.	✓

All ten test cases passed on the live deployment. The agent demonstrated correct handling of the full booking lifecycle, including error paths (OCR rejection, slot conflicts, hold expiry, vendor rejection).

5.4 Discussion

5.4.1 Key Findings

1. **Compact models outperform large multilingual models on small domain-specific datasets.** DistilBERT (66M parameters) achieves 90.5% accuracy vs XLM-RoBERTa's (270M parameters) 70.5%. Domain-specific fine-tuning on 657 utterances contributes more than additional model capacity.

2. **The Neuro-Symbolic architecture effectively prevents hallucination.** By routing all database mutations through deterministic code nodes, the system eliminates the class of errors where agents "hallucinate" bookings or bypass payment verification. Zero such failures were observed across 10 structured test cases.

3. **Roman Urdu code-switching is manageable with targeted augmentation.** Despite severe orthographic variation, the DistilBERT model trained on the augmented bilingual dataset correctly handled "padel kal" (padel tomorrow), "raat" (night), and other code-switched time references in all test cases.

5.4.2 Limitations

- **Dataset size:** 694 samples is sufficient for a 5-class intent classification baseline but insufficient for robust slot extraction across all Roman Urdu phonetic variations. Expanding to 2,000+ samples would improve generalisation.

- **Latency on free-tier deployment:** Response times of 38–52 seconds, while within the 60-second NFR target, would benefit from caching and a paid LLM API endpoint. Production infrastructure would reduce this to under 10 seconds.

- **transaction_confirm confusion:** 18% misclassification into `inquiry` requires the dialogue state layer as a safety net. Targeted additional training data for this class boundary would address it.

6. Conclusion and Future Work

6.1 Summary

This report presented **BookForMe**, a centralized, agentic AI-powered booking platform designed to address the fragmented, manual booking process for informal services in Karachi, Pakistan. The platform serves three user groups—app customers, WhatsApp customers, and vendors—through two interaction modalities: a React Native mobile application and an AI Receptionist operating on the vendor's WhatsApp Business channel.

The core technical contributions are:

1. A **custom bilingual dataset** of 694 annotated English / Roman Urdu booking utterances, constructed through real vendor conversations, manual annotation, and LLM-driven data augmentation.

2. A **comparative NLU evaluation** of six transformer architectures, establishing that fine-tuned DistilBERT achieves the best accuracy–latency tradeoff (90.5% accuracy, 91.7% macro-F1, ≈47 ms inference on CPU) for this domain.

3. A **production-deployed LangGraph agent** implementing the full booking pipeline with Optimistic Concurrency Control for double-booking prevention, OCR-based payment verification, and human-in-the-loop vendor approval.

4. A **full-stack social platform** with Elo-based matchmaking, community forums, leaderboards, and direct messaging—all integrated with the core booking infrastructure.

End-to-end evaluation across 10 structured test cases demonstrated correct agent behaviour for the complete booking lifecycle, including error paths (incorrect payment, expired holds, slot conflicts, vendor rejection). All non-functional requirements—including performance, security, reliability, and usability—were met on the live deployment.

6.2 Research Questions Revisited

RQ1: Centralised platform vs. fragmented manual systems The deployed platform eliminates the three-step manual booking cycle (find number → wait for reply → confirm informally) by providing a single interface for discovery, booking, and payment across categories. Vendor survey feedback confirmed reduced scheduling overhead.

RQ2: Bilingual NLU on a small custom dataset DistilBERT fine-tuned on 657 utterances achieves 90.5% accuracy and 91.7% macro-F1 on the held-out test set, demonstrating that production-grade bilingual NLU is achievable with targeted dataset curation and augmentation—without requiring thousands of manually labelled examples.

RQ3: Conversational coherence across asynchronous WhatsApp interactions The LangGraph cyclic state graph, with conversation state persisted in Firestore, correctly handles mid-conversation context switches, late replies, and multi-turn clarification loops in all evaluated test cases.

RQ4: Double-booking prevention in a distributed multi-channel platform Optimistic Concurrency Control via Firestore atomic transactions successfully prevents concurrent slot conflicts, as demonstrated in TC-08 where two simultaneous booking requests for the same slot resulted in exactly one confirmed booking and one conflict response with alternatives.

6.3 Future Work

Building on the current platform, the following directions are planned or proposed:

Voice Agent (Phase 2) Extending the AI Receptionist to handle phone calls via Twilio Voice with Whisper ASR (speech-to-text) and a TTS response, enabling the estimated 30% of potential users who prefer voice communication over text.

Production WhatsApp Re-Integration Obtaining a production WhatsApp Business API account to replace the expired Twilio sandbox number, enabling evaluation with real vendor traffic at scale.

Payment Gateway Integrating JazzCash or EasyPaisa payment APIs to replace screenshot-based payment verification with programmatic confirmation, eliminating the OCR accuracy dependency and reducing the booking cycle from $\sim$52 seconds to under 20 seconds.

Expanded NLU Dataset Growing the bilingual dataset to 2,000+ samples using active learning (the deployed agent auto-labels new conversations for human review) and extending coverage to more vendor categories (clinics, tutoring centres, event halls).

Advanced Matching Algorithms Replacing the current Elo system with Glicko-2 (which models rating uncertainty) and implementing graph-based player matching (stable matching, max-flow) for fairer lobby formation with asymmetric skill distributions.

Recommendation Engine Collaborative filtering and link prediction on the social graph to generate personalised venue suggestions based on booking history, friend activity, and sport preferences.

Cross-City Expansion Extending the vendor database to Lahore and Islamabad, requiring location-aware search indexing and multi-city availability management.

Algorithmic Fairness Auditing Evaluating the matchmaking and recommendation systems for demographic fairness (e.g., equitable match quality across skill levels), following emerging best practices in fairness-aware ML deployment.

7. Reflection

7.1 Learning Reflection

7.1.1 Muhammad Ahmad Humayun Hanif

Kaavish was the first project in which I was responsible for the complete system architecture of a production-grade application—not just a module, but the integration of an LLM pipeline, a distributed database, a WhatsApp API, an OCR service, and a mobile frontend into a coherent whole. The most significant skill I gained was **system-level thinking**: understanding how a design decision in the data model (e.g., separating Payments from Bookings) has downstream consequences for the agent's OCR verification loop, the vendor dashboard UI, and the audit trail.

Working with LangGraph taught me that *cyclic state* is fundamentally different from linear pipeline composition—debugging a loop that can re-enter a node after five asynchronous hops requires a completely different mental model than debugging a sequential program. I also developed a deep appreciation for the Neuro-Symbolic design pattern: the security and reliability properties it provides (no hallucinated bookings, no prompt injection) are not achievable with a pure LLM approach.

The hardest challenge was the Roman Urdu orthographic variation problem. I had underestimated how much "waqt" versus "tym" versus "vakt" would matter for a tokeniser trained on formal text. Solving it through targeted data augmentation rather than a more complex normalisation layer was a pragmatic decision I stand by, but it highlighted the importance of domain-specific data over model complexity.

7.1.2 Jazib Waqas

Designing and implementing the backend was simultaneously the most technical and the most frustrating part of the project. The Optimistic Concurrency Control implementation in Firestore was conceptually straightforward but took significant debugging to get right—particularly understanding that Firestore transactions retry automatically on conflict, which required careful idempotency guarantees in our booking creation code to prevent duplicate records on retry.

I learned the value of **API contract discipline**: establishing clear input/output schemas for each microservice (agent, OCR, booking APIs) early in the project prevented integration failures later. The one week we spent without a formal API contract between the backend and frontend resulted in two weeks of alignment work afterwards.

WhatsApp Business API integration was technically straightforward but operationally unpredictable: Meta's review processes for webhook verification and message template approval add real latency to development cycles. Future projects with third-party communication APIs should budget this time explicitly.

7.1.3 Taha Hunaid Ali

Building the React Native frontend taught me that **state management at scale** is a qualitatively different problem from small-project UI development. Coordinating real-time Firestore listeners, local optimistic UI updates (showing a slot as booked before the OCC transaction completes), and background push notification handling required careful architecture—I rewrote the booking flow's state management twice before arriving at a design that was both correct and maintainable.

The social features (forum, leaderboard, chat) were the most enjoyable to build because they had immediate visual feedback and were the most enthusiastically received by test users. They also taught me the importance of **performance budgeting** on mobile: unoptimised Firestore listeners can drain a phone's battery within hours, and pagination on the leaderboard query reduced initial load time from 8 seconds to under 2 seconds.

7.1.4 Muhammad Taqi

My primary learning from this project was about **the cost of data**. We spent more calendar time constructing, annotating, and augmenting the bilingual dataset than on any other single task. This experience permanently reframed my view of ML projects: the model choice matters far less than the quality and coverage of training data. The 20-point accuracy gap between XLM-RoBERTa (which should theoretically be better at multilingual tasks) and DistilBERT on our small custom dataset makes this case compellingly.

Fine-tuning transformers also taught me practical lessons that textbooks gloss over: focal loss is essential for class imbalance but its gamma parameter requires careful tuning; evaluation should use macro-F1 (not accuracy) when classes are imbalanced; and confidence calibration matters when the downstream system (the dialogue state validator) relies on model confidence to decide whether to ask a clarifying question.

7.2 Team Reflection

7.2.1 Collaboration and Role Distribution

The team operated with a flat, ownership-based model: each member owned specific modules end-to-end (backend APIs, NLU pipeline, frontend, agent), rather than collectively owning all code. This accelerated individual progress but created integration risks that required dedicated "integration weeks" at the end of each phase.

In hindsight, establishing a shared API contract document at the project's start—and treating it as a living specification updated before any breaking change—would have reduced integration friction significantly. We introduced this practice in Kaavish II after experiencing its absence in Kaavish I.

7.2.2 Teamwork Challenges

The most significant teamwork challenge arose when the WhatsApp Twilio sandbox number expired two weeks before the mid-evaluation, forcing an emergency re-

integration. The team's ability to divide the problem—one member handling the new Twilio credentials, another updating the webhook endpoint, a third preparing a fallback demo with the web chat interface—under time pressure was a genuine test of collaboration and emerged as one of our team's strengths.

7.3 Project Reflection

7.3.1 Proposed vs. Achieved

Table 7.1 compares the features proposed in the original Kaavish I proposal against what was actually delivered.

Table 7.1: Proposed vs. achieved features.

Feature	**Proposed**	**Achieved**
WhatsApp AI Receptionist (text-based)	✓	✓
Bilingual NLU (English + Roman Urdu)	✓	✓
OCC slot locking (no double-booking)	✓	✓
OCR payment verification	Stretch	✓
React Native mobile app	✓	✓
Vendor dashboard with analytics	✓	✓
Social features (forum, chat, friends)	✓	✓
Elo-based matchmaking	✓	✓
Voice-calling agent (Twilio Voice)	Stretch	✗
Live payment gateway (JazzCash)	Stretch	✗
Google Sheets sync	✓	Partial
Multi-city expansion	Stretch	✗

OCR payment verification was a stretch goal that we implemented ahead of schedule, replacing it in the core pipeline. Voice calling and the live payment gateway were deprioritised in favour of polishing the text-based WhatsApp agent and the mobile app experience—a deliberate decision based on the user survey finding that 72% of target users primarily communicate via WhatsApp text.

7.3.2 Design Decisions Driven by Challenges

- **Neuro-Symbolic over pure LLM:** Initially planned to use a single LLM for both NLU and database queries. After observing the LLM generating invalid Firestore queries and "inventing" booking confirmations in early prototypes, we adopted the Neuro-Symbolic architecture where the LLM only classifies intent and extracts entities.
- **DistilBERT over Qwen2.5:** Originally planned to use Qwen2.5-7B as the NLU backbone. After evaluating inference latency (>3 seconds on available GPU, exceeding our WhatsApp response budget), we switched to fine-tuning smaller transformer models, with DistilBERT emerging as the winner.
- **Firestore over PostgreSQL:** The SRS originally listed PostgreSQL as the primary database. After evaluating real-time listener requirements for the social layer (chat, notifications, leaderboard) and the serverless OCC transaction support needed for the booking pipeline, we migrated to Firestore. This decision eliminated a planned data synchronisation layer.

7.4 Process Reflection

7.4.1 What Worked Well

- **Phased development:** Delivering a minimal working agent (text only, limited intents) in December 2025 and iterating from there gave us early feedback and a working demo for stakeholders at every subsequent milestone.
- **Structured test cases:** Maintaining a public Google Sheets test case document throughout development (rather than writing tests post-hoc) caught agent regression bugs early and gave us clear evidence for the mid-evaluation presentation.

- **Real vendor engagement:** Testing the agent with actual futsal operators—who sent genuine booking messages in Roman Urdu—surfaced orthographic variations and failure modes that synthetic data generation did not cover.
- **Early SRS/SDS discipline:** Writing the SRS and SDS before coding prevented scope creep and gave every team member a shared vocabulary for system components.

7.4.2 What Could Be Improved

- **API contract documentation:** As noted above, a formal shared API schema document from day one would have reduced integration friction.
- **Earlier load testing:** We discovered the Render.com free-tier cold-start latency issue (≈8–12 seconds) only during the mid-evaluation rehearsal. Load testing in January would have given us time to implement response caching before the demo.
- **Dataset annotation tooling:** Manual annotation in a shared Google Sheet was slow and error-prone for the Roman Urdu samples. A proper annotation tool (Label Studio, Prodigy) with inter-annotator agreement tracking would have improved dataset quality and annotation speed.
- **Third-party API contingency planning:** The WhatsApp Twilio sandbox number expiry was avoidable with a simple API credential rotation schedule. Future projects should treat API credential expiry as a first-class risk with a documented mitigation plan.

References

[1] Ziqiang An et al. "Controllable Data Augmentation for Few-Shot Spoken Language Understanding". In: *Proceedings of INTERSPEECH 2022*. 2022, pp. 4182–4186. URL: https://www.isca-archive.org/interspeech_2022/an22_interspeech.html.

[2] Irshad Ahmad Bhat et al. *A Survey of Code-Mixed NLP: Methods and Challenges*. 2023. URL: https://arxiv.org/abs/2510.07037.

[3] BookForMe Team. *Booking Preferences User Survey*. 2025. URL: https://docs.google.com/forms/d/1CPqAgHxeoLK3ZT554aPRTWvKXI1P3vXpdoZwGApsr4.

[4] Sanjukta Chatterjee et al. "Turf Booking System: A Study on Real-Time Slot Availability and Automated Conflict Resolution". In: *International Journal of Futuristic Multidisciplinary Research* (2025). URL: https://www.ijfmr.com/papers/2025/3/44829.pdf.

[5] Zhiyu Chen et al. "Adaptive Retrieval-Augmented Generation for Conversational Systems". In: *Findings of the Association for Computational Linguistics: NAACL 2025*. 2025. URL: https://aclanthology.org/2025.findings-naacl.30.pdf.

[6] Alexis Conneau et al. *Unsupervised Cross-Lingual Representation Learning at Scale*. 2020. URL: https://arxiv.org/abs/1911.02116.

[7] Jacob Devlin et al. *BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding*. 2019. URL: https://arxiv.org/abs/1810.04805.

[8] Arpad E. Elo. *The Rating of Chessplayers, Past and Present*. New York: Arco, 1978.

[9] Thorsten Händler. *Balancing Autonomy and Alignment: A Multi-Dimensional Taxonomy for Autonomous LLM-Powered Multi-Agent Architectures*. 2023. URL: https://arxiv.org/abs/2310.03659.

[10] Weijie He et al. "SynthDST: Synthetic Data is All You Need for Few-Shot Dialog State Tracking". In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 2024, pp. 11658–11673. URL: https://aclanthology.org/2024.emnlp-main.684/.

[11] Ehsan Hosseini-Asl et al. *A Simple Language Model for Task-Oriented Dialogue*. 2020. URL: https://arxiv.org/abs/2005.00796.

[12] Yutai Hou et al. "Few-Shot Slot Tagging with Collapsed Dependency Transfer and Label-Enhanced Task-Adaptive Projection Network". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 2020, pp. 1381–1393. URL: https://aclanthology.org/2020.acl-main.127/.

[13] R. Kesavamoorthy et al. "Online Booking System Using AI and Cloud Integration". In: *International Research Journal of Engineering and Technology* 7.6 (2020). URL: https://www.irjet.net/archives/V7/i6/IRJET-V7I6542.pdf.

[14] LangChain, Inc. *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. 2024. URL: https://langchain-ai.github.io/langgraph/.

[15] Patrick Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. 2020. URL: https://arxiv.org/abs/2005.11401.

[16] Zhiyuan Liu et al. *PRACT: Optimizing Principled Reasoning and Acting of LLM Agents*. 2024. URL: https://arxiv.org/abs/2410.18528.

[17] Vasilios Mavroudis. *LangChain*. 2024. DOI: 10.20944/preprints202411.0566.v1.

[18] MoldStud. *Increasing Efficiency with Automated Booking and Reservation Systems*. 2024. URL: https://moldstud.com/articles/p-increasing-efficiency-with-automated-booking-and-reservation-systems.

[19] M. S. N. Mong'are. *Understanding Idempotency in APIs and Distributed Systems*. 2024. URL: https://dev.to/msnmongare/understanding-idempotency-in-apis-and-distributed-systems-3afb.

[20] Shishir G. Patil et al. *Gorilla: Large Language Model Connected with Massive APIs*. 2023. URL: https://arxiv.org/abs/2305.15334.

[21] Yujia Qin et al. *ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs*. 2023. URL: https://arxiv.org/abs/2307.16789.

[22] Qwen Team. *Qwen2.5 Technical Report*. 2025. URL: https://arxiv.org/abs/2412.15115.

[23] Victor Sanh et al. *DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter*. 2019. URL: https://arxiv.org/abs/1910.01108.

[24] Timo Schick et al. *Toolformer: Language Models Can Teach Themselves to Use Tools*. 2023. URL: https://arxiv.org/abs/2302.04761.

[25] Yuchen Shang et al. *AgentSquare: Automatic LLM Agent Search in Modular Design Space*. 2024. URL: https://arxiv.org/abs/2410.06153.

[26] Jamin Shin et al. "Dialogue Summaries as Dialogue States (DS2): Template-Guided Summarization for Few-Shot Dialogue State Tracking". In: *Findings of the Association for Computational Linguistics: ACL 2022*. 2022. URL: https://aclanthology.org/2022.findings-acl.302/.

[27] Noah Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. 2023. URL: https://arxiv.org/abs/2303.11366.

[28] Satinder Singh et al. "Optimizing Dialogue Management with Reinforcement Learning: Experiments with the NJFun System". In: *Journal of Artificial Intelligence Research* 13 (2000), pp. 105–124. URL: https://web.eecs.umich.edu/~baveja/Papers/RLDSjair.pdf.

[29] Lei Wang et al. "A Survey on Large Language Model Based Autonomous Agents". In: *Frontiers of Computer Science* (2024). DOI: 10.1007/s11704-024-40231-1.

[30] Qingyun Wu et al. *AutoGen: Enabling Next-Generation LLM Applications via Multi-Agent Conversation*. 2023. URL: https://arxiv.org/abs/2308.08155.

[31] Yinpeng Wu et al. "Towards Zero-Shot Multilingual Transfer for Code-Switched Responses: An Adapter-Based Cross-Lingual Framework". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 2023. URL: https://aclanthology.org/2023.acl-long.417/.

[32] Zhiheng Xi et al. *The Rise and Potential of Large Language Model Based Agents: A Survey*. 2023. URL: https://arxiv.org/abs/2309.07864.

[33] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. 2023. URL: https://arxiv.org/abs/2210.03629.

[34] Dian Yu, Zhou Yu, and Kenji Sagae. "Cross-Lingual Dialogue State Tracking via Contrastive Learning". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023. URL: https://aclanthology.org/2023.ccl-1.54/.